gates Manual

A small retained GUI library in C23 for tool-style Windows programs

Version 0.15.2

https://github.com/rubidus-api/gates_gui_lib

The English edition is the original.
MIT License.

Contents

0. Start here 1
- 0.1 What gates is for 1
- 0.2 The package 1
- 0.3 The first program 1
- 0.4 Without a window 3
- 0.5 The headers 3

1. Concepts 5
- 1.1 The node tree 5
- 1.2 Ownership: copied or borrowed 6
- 1.3 Errors 6
- 1.4 Events and safe points 6
- 1.5 Units 7

2. Controls and layout 8
- 2.1 The control matrix 8
- 2.2 Layout 10

3. Forms 12
- 3.1 A form holds no data 12
- 3.2 Draft validation 13

4. Commands, focus, dialogs and menus 14
- 4.1 One action, many ways to ask for it 14
- 4.2 Focus 15
- 4.3 Dialogs 15
- 4.4 Context menus 15
- 4.5 Undo and redo 15

5. Views over your data 18
- 5.1 One node for any number of rows 18
- 5.2 Lists, tables, trees and logs 20
- 5.3 Cells people can change 20
- 5.4 Many rows at once: multi-selection 23
- 5.5 A record's fields: the property grid 25
- 5.6 Rules for models 26

6. Text entry 27
- 6.1 The text box 27
- 6.2 The limit is a question 27
- 6.3 The installed input method 28
- 6.4 Unicode limits in 0.11.0 28
- 6.5 The clipboard 29
- 6.6 Other encodings at the edge 29
- 6.7 Many lines: the editor 31
- 6.8 Large texts 33

7. Workers and timers 34
- 7.1 Workers never touch the tree 34
- 7.2 Background tasks 36
- 7.3 Timers 38

8. Themes, units and scaling 39
- 8.1 Logical units 39
- 8.2 Themes 39
- 8.3 Fonts 40
- 8.4 Drawing and text backends 40

9. Accessibility 42
9.1 Every window is readable and usable without sight or pointer 42
9.2 Names 42
9.3 The enforced rules 43
9.4 Speaking up 43
9.5 Checking with Narrator 43
10. Deployment and troubleshooting 44
10.1 Linking 44
10.2 Versions and compatibility 44
10.3 The application and its windows 44
10.4 Troubleshooting 45
11. The application frame 46
11.1 Mnemonics 46
11.2 The menu bar 46
11.3 Keymap: listing and rebinding shortcuts 47
11.4 Toolbar 48
11.5 Status bar 49
11.6 Tooltips 49
11.7 Tabs 51
11.8 Keeping the arrangement 51
11.9 Accessibility 52
12. Numbers, groups and layouts 53
12.1 One handler for many controls 53
12.2 Once per turn: deferred calls 53
12.3 Spin box and slider 53
12.4 Group box 55
12.5 Grid and wrap layouts 55
12.6 Accessibility 56
13. Images and native dialogs 57
13.1 Images 57
13.2 Icons 57
13.3 Native dialogs 59

0 Start here

In this chapter

0.1 What gates is for 1
0.2 The package 1
0.3 The first program 1
0.4 Without a window 3
0.5 The headers 3

0.1 What gates is for

gates is a GUI library for programs whose job is to let a person look at something and change it: a settings dialog, a record inspector, a log viewer, a build tool. It is written in C23, keeps a tree of nodes that describes the interface, and does the work that such programs otherwise redo badly: keyboard focus, text entry with the installed input method, forms, lists over large data, commands shared by buttons and shortcuts, dialogs, threads that feed the interface, themes, scaling, and accessibility.

It is not a browser engine, not a game UI, and not a pixel-exact drawing kit. Think of HTML and CSS: the program says what things are and how they relate; how they finally look is the business of the backend and the theme. A feature in gates is finished when the interaction it serves works - reachable, usable with pointer, keyboard and input method, readable by a screen reader - not when it matches a picture.

Version 0.15.2 has one backend: Win32 with a software renderer. The core (everything except the window) is platform-free and runs anywhere a C23 compiler does, which is how its tests run.

0.2 The package

A release is a folder, `gates-0.15.2/`:

Path	What
`include/gates/`	the public headers; `gates/gates.h` includes all of them
`include/proven/`	the headers of the proven library that gates builds on
`lib/win64/libgates.a`	gates for Windows (core + Win32 backend), mingw-w64
`lib/host/libgates_core.a`	the platform-free core, for tests and headless tools
`lib/*/libproven.a`	proven, the foundation library (allocators, strings, results)
`manual/`, `manual-ko/`	this manual in English and Korean
`examples/consumer/`	a program built from the package alone, with its Makefile

gates and proven are separate libraries: link both, gates first. A program that already uses proven (0.1.x) may link its own copy instead of the one in the package.

0.3 The first program

A window with a label and a button; pressing the button changes the label.

```
/* manual example (windows): the first program - a window, a label, a button.
 * Build: see chapter 0 (links -lgates -lproven and the Windows libraries). */
```

```c
#include <gates/gates.h>

#include <stdio.h>

typedef struct hello_t {
    gates_app_t *app;
    gates_tree_t *tree;
    gates_node_t count_label;
    int clicks;
} hello_t;

/* The button tells us it was pressed; we change the label. Nothing is read
 * back from widgets while painting. */
static void on_button(gates_tree_t *tree, const gates_event_t *ev, void *user) {
    hello_t *h = user;
    if (ev->kind != GATES_EVENT_ACTIVATED) return;
    h->clicks++;
    char text[48];
    int n = snprintf(text, sizeof text, "Pressed %d time%s", h->clicks, h->clicks == 1 ? "" :
"s");
    (void)gates_widget_set_text(tree, h->count_label, (gates_str_t){ .ptr = (const gates_u8
*)text, .size = (gates_usize_t)n });
}

static void on_key(gates_window_t *win, const gates_key_event_t *ev, void *user) {
    (void)win;
    hello_t *h = user;
    if (ev->down && ev->key == GATES_KEY_ESCAPE) gates_app_quit(h->app);
}

int main(void) {
    gates_app_t *app = nullptr;
    if (!gates_is_ok(gates_app_create(&(gates_app_desc_t){0}, &app))) return 1;
    hello_t h = { .app = app };
    gates_window_desc_t desc = { .title = GATES_STR("Hello, gates"), .size = { 320, 160 } };
    gates_window_callbacks_t cb = { .on_key = on_key, .user_data = &h };
    gates_window_t *win = nullptr;
    if (!gates_is_ok(gates_window_create(app, &desc, &cb, &win))) {
        gates_app_destroy(app);
        return 1;
    }
    h.tree = gates_window_tree(win);
    gates_node_t root = gates_tree_root(h.tree), button;
    gates_err_t err = gates_layout_set(h.tree, root, GATES_LAYOUT_KIND_COLUMN);
    if (gates_is_ok(err)) err = gates_layout_set_padding(h.tree, root, 12);
    if (gates_is_ok(err)) err = gates_label_create(h.tree, root, GATES_STR("Not pressed yet"),
&h.count_label);
    if (gates_is_ok(err)) err = gates_button_create(h.tree, root, GATES_STR("Press me"),
nullptr, nullptr, &button);
    if (gates_is_ok(err)) err = gates_widget_set_handler(h.tree, button, on_button, &h);
    if (gates_is_ok(err)) err = gates_app_run(app); /* returns when the last window closes */
    gates_window_destroy(win);
    gates_app_destroy(app);
    return gates_is_ok(err) ? 0 : 1;
}
```

Build it with mingw-w64 (on Linux, or MSYS2 on Windows), from the package folder:

```
x86_64-w64-mingw32-gcc -std=c23 -O2 -Iinclude hello.c -Llib/win64 -lgates -lproven \
    -lgdi32 -luser32 -limm32 -ldwmapi -ladvapi32 -luiautomationcore -lole32 -loleaut32 \
    -luuid -mwindows -o hello.exe
```

Three things in it are the whole shape of a gates program. The program builds nodes into the window's tree (`gates_label_create`, `gates_button_create`). It learns what the person did from events (`gates_widget_set_handler`), never by reading widgets while the window paints. And `gates_app_run` owns the loop until the last window closes.

0.4 Without a window

The core needs no window: build a tree, lay it out with the builtin text backend, and ask what is where. Tests and command-line tools use gates this way.

```
/* manual example (host): gates without a window - build, lay out, inspect.
 * expect: Press me: button at 0,16 320x26 */
#include <gates/gates.h>

#include <stdio.h>

int main(void) {
    gates_tree_t *tree = nullptr;
    if (!gates_is_ok(gates_tree_create(&(gates_tree_desc_t){0}, &tree))) return 1;
    gates_node_t root = gates_tree_root(tree), label, button;
    gates_err_t err = gates_layout_set(tree, root, GATES_LAYOUT_KIND_COLUMN);
    if (gates_is_ok(err)) err = gates_label_create(tree, root, GATES_STR("Not pressed yet"),
&label);
    if (gates_is_ok(err)) err = gates_button_create(tree, root, GATES_STR("Press me"), nullptr,
nullptr, &button);
    /* Layout needs text metrics: the builtin backend works everywhere. */
    if (gates_is_ok(err)) err = gates_layout_run(tree, (gates_size_t){ 320, 160 },
gates_text_backend_builtin());
    gates_access_info_t info;
    if (gates_is_ok(err)) err = gates_access_info(tree, button, 0, &info);
    if (gates_is_ok(err)) {
        printf("%.*s: %s at %d,%d %dx%d\n", (int)info.name.size, (const char *)info.name.ptr,
               info.role == GATES_ROLE_BUTTON ? "button" : "?", info.bounds.x, info.bounds.y,
info.bounds.w,
               info.bounds.h);
    }
    gates_tree_destroy(tree);
    return gates_is_ok(err) ? 0 : 1;
}
```

```
cc -std=c23 -Iinclude headless.c -Llib/host -lgates_core -lproven -lm -o headless
```

It prints `Press me: button at 0,16 320x26`: the column put the button under the label and stretched it to the full width.

0.5 The headers

`gates/gates.h` includes everything. The headers, in the order this manual meets them:

Header	What it holds	Chapter
`gates/types.h`	integers, strings (`gates_str_t`), errors, the allocator	1
`gates/version.h`	the version and `gates_version()`	10
`gates/tree.h`	the node tree, handles, focus, hidden nodes	1
`gates/event.h`	typed events and their delivery	1
`gates/widget.h`	labels, buttons, check boxes, text boxes, radio groups, choices, progress	2
`gates/layout.h`	row, column, stack, split, scroll, form layouts	2
`gates/geometry.h`	points, sizes, rectangles, logical units	2, 8

Header	What it holds	Chapter
`gates/form.h`	forms with labels, help and error lines	3
`gates/command.h`	commands shared by buttons, shortcuts and menus	4
`gates/overlay.h`	modal dialogs and context menus	4
`gates/view.h`	lists, tables, trees and logs over your data	5
`gates/text_edit.h`	the text edit core under every text box	6
`gates/clipboard.h`	the clipboard boundary	6
`gates/input.h`	pointer and key events	6
`gates/ui.h`	input routing, hit testing, painting a tree	6
`gates/post.h`	posting from worker threads	7
`gates/timer.h`	timers on the UI thread	7
`gates/theme.h`	themes: colour tokens, light, dark, high contrast	8
`gates/draw.h`	the draw command list	8
`gates/render.h`	the software renderer	8
`gates/text.h`	fonts, text metrics and text backends	8
`gates/access.h`	accessibility: roles, names, actions, the audit	9
`gates/app.h`	the application and its loop	10
`gates/window.h`	windows, themes, zoom, announcements	10

1 Concepts

In this chapter

1.1 The node tree 5
1.2 Ownership: copied or borrowed 6
1.3 Errors 6
1.4 Events and safe points 6
1.5 Units 7

Headers: `gates/tree.h`, `gates/types.h`, `gates/event.h`.

1.1 The node tree

Everything a window shows is a node in its tree: panels that group, labels, buttons, text boxes, whole views. The program creates nodes, puts them under parents, and changes them; gates lays them out, paints them, routes input to them, and describes them to assistive technology. The tree is retained: the program does not redraw anything by hand, it changes the tree and the window repaints what changed.

A node is named by a handle, `gates_node_t`: a slot index and a generation. When a node is destroyed its slot can later hold another node with a new generation, so an old handle never reaches the new node - every call checks, and a stale handle is refused (`PROVEN_ERR_INVALID_ARG`), never obeyed. Destroying marks a whole subtree at once; the slots are freed at a safe point (`gates_tree_flush_destroys`, which a window calls after each input turn), so a handler can destroy the node that sent its event.

```
/* manual example (host): handles, generations and copies.
 * expect: stale handle refused; label kept its copy */
#include <gates/gates.h>

#include <stdio.h>
#include <string.h>

int main(void) {
    gates_tree_t *tree = nullptr;
    if (!gates_is_ok(gates_tree_create(&(gates_tree_desc_t){0}, &tree))) return 1;
    gates_node_t root = gates_tree_root(tree), old, fresh;

    /* Text is copied: the caller's buffer may change or go away at once. */
    char buf[16] = "first";
    if (!gates_is_ok(gates_label_create(tree, root, (gates_str_t){ .ptr = (const gates_u8
*)buf, .size = strlen(buf) }, &old))) return 1;
    strcpy(buf, "XXXXX");

    /* Destroying marks the node; its slot is freed at a safe point. */
    if (!gates_is_ok(gates_node_destroy(tree, old))) return 1;
    (void)gates_tree_flush_destroys(tree);

    /* A new node may reuse the slot, with a new generation: the old handle
     * never reaches it. */
    if (!gates_is_ok(gates_label_create(tree, root, GATES_STR("second"), &fresh))) return 1;
    bool stale_refused = !gates_node_is_valid(tree, old) &&
```

```
                        gates_widget_set_text(tree, old, GATES_STR("oops")) ==
PROVEN_ERR_INVALID_ARG;
    gates_str_t now = gates_widget_text(tree, fresh);
    bool kept = now.size == 6 && memcmp(now.ptr, "second", 6) == 0;
    printf("%s; %s\n", stale_refused ? "stale handle refused" : "STALE HANDLE WORKED",
           kept ? "label kept its copy" : "label changed");
    gates_tree_destroy(tree);
    return stale_refused && kept ? 0 : 1;
}
```

1.2 Ownership: copied or borrowed

Every string the program gives gates is copied when the call returns successfully: labels, texts, command labels, option labels, names. The caller's buffer is free to change at once. What gates gives back is borrowed:

Returned	Valid until
`gates_widget_text`, `gates_textbox_text`	the next change of that node
`ev->text` in an event	the handler returns
a view cell (`gates_cell_t.text`, from your model)	the next call into your model
`gates_access_info_t` strings	the next `gates_access_info` call on the tree, or any change

Callbacks and `user` pointers registered with gates (handlers, models, command functions) are borrowed for as long as they are registered: keep them alive, or unregister first.

1.3 Errors

Functions that can fail return `gates_err_t` and are marked `[[nodiscard]]`. `GATES_OK` (test it with `gates_is_ok`) means the whole change happened; any other value means none of it did - a node that could not be created leaves nothing half-built behind. The codes are proven's: `PROVEN_ERR_INVALID_ARG` (a bad or stale argument), `PROVEN_ERR_INVALID_STATE` (not now: a disabled control, a closed dialog), `PROVEN_ERR_NOMEM`, `PROVEN_ERR_OUT_OF_BOUNDS` (a limit). Out of memory is an ordinary answer, not a crash: the reference applications handle it. Input that cannot fail loudly (a key, a click) records its failure for `gates_input_take_error`; a window's `on_input_error` callback hears it after each input turn, and the reference applications show it in their status line.

1.4 Events and safe points

A program learns what the person did through events. Each widget has at most one handler (`gates_widget_set_handler`); when the person changes something, gates queues an event describing it, and the window delivers the queue at a safe point after the input message has been handled (`gates_tree_dispatch_events`). Nothing is delivered in the middle of an input operation, so a handler may change or destroy anything, including its source.

Changes the program makes itself are silent: it already knows. When the rest of the program should hear about one, `gates_widget_notify` queues an event with origin `GATES_ORIGIN_PROGRAM`.

```
/* manual example (host): events - what the person did, delivered at a safe point.
 * expect: user checked=1, program checked=0 */
#include <gates/gates.h>

#include <stdio.h>

static char log_text[128];
```

```
static int log_len;

static void on_value(gates_tree_t *tree, const gates_event_t *ev, void *user) {
    (void)tree;
    (void)user;
    if (ev->kind != GATES_EVENT_VALUE_CHANGED) return;
    log_len += snprintf(log_text + log_len, sizeof log_text - (size_t)log_len, "%s%s
checked=%d",
                        log_len > 0 ? ", " : "", ev->origin == GATES_ORIGIN_USER ? "user" :
"program",
                        ev->checked ? 1 : 0);
}

int main(void) {
    gates_tree_t *tree = nullptr;
    if (!gates_is_ok(gates_tree_create(&(gates_tree_desc_t){0}, &tree))) return 1;
    gates_node_t box;
    if (!gates_is_ok(gates_checkbox_create(tree, gates_tree_root(tree), GATES_STR("Send news"),
false, nullptr, nullptr, &box)) ||
        !gates_is_ok(gates_widget_set_handler(tree, box, on_value, nullptr))) {
        return 1;
    }
    /* A person ticks the box (here through the accessibility action, which
     * takes the same path as a click): an event is queued, not delivered. */
    if (!gates_is_ok(gates_access_toggle(tree, box))) return 1;
    /* The window delivers queued events after each input message; without a
     * window, the program does it. */
    (void)gates_tree_dispatch_events(tree, 0);

    /* Setters are silent: the program knows what it did. Announce it
     * explicitly when the rest of the program should hear it. */
    if (!gates_is_ok(gates_checkbox_set_checked(tree, box, false))) return 1;
    if (!gates_is_ok(gates_widget_notify(tree, box))) return 1;
    (void)gates_tree_dispatch_events(tree, 0);

    printf("%s\n", log_text);
    gates_tree_destroy(tree);
    return 0;
}
```

Two rules follow. Never read widget state while painting to find out what changed - follow events. And never change a model from inside a paint callback; paint only draws.

1.5 Units

Every coordinate and size in the API is a logical unit, 1/96 inch. A window converts once, at its edge. Chapter 8 has the details; until then, read "units" where you would expect pixels.

2 Controls and layout

In this chapter

2.1 The control matrix 8
2.2 Layout 10

Headers: `gates/widget.h`, `gates/layout.h`, `gates/geometry.h`.

2.1 The control matrix

Every control is one node. The table is the whole finite set in 0.15.2: what the person does with it, the keys, the events the program receives, and what a screen reader hears (chapter 9).

Control	Create	The person	Keys	Events	Accessible as
panel	`gates_panel_create`	sees a group	-	-	nothing, or a group when named
label	`gates_label_create`	reads text	-	-	text
button	`gates_button_create`	asks for one action	Space (on release), Enter	ACTIVATED	button (Invoke)
check box	`gates_checkbox_create`	turns an option on or off	Space	VALUE_CHANGED	Check box (Toggle)
text box	`gates_textbox_create`	types one line	editing keys, Ctrl+C/X/V/Z/Y, the IME; double click a word, triple click all, Shift+click extends	TEXT_CHANGED, PREEDIT_CHANGED, LIMIT_EXCEEDED	Edit (Value, Text)
editor	`gates_editor_create`	writes many lines	editing keys, Ctrl+arrows, PageUp/PageDown, Ctrl+C/X/V/Z/Y, the IME; Tab when asked; double click a word, triple click a line, Shift+click extends	TEXT_CHANGED, SELECTION_CHANGED, PREEDIT_CHANGED	Edit (Value, TextEd, line) (Chapter 6)
radio group	`gates_radio_create`	picks one of a few	arrows, Home, End, Space	VALUE_CHANGED (result = option id)	Group of radio buttons
choice	`gates_choice_create`	picks one from a list	Space, Enter, Alt+Down open; arrows, Enter, Escape	VALUE_CHANGED	Combo box (Selection, Expand)
progress	`gates_progress_create`	sees how far work has come	-	-	progress bar (percent)

Control	Create	The person	Keys	Events	Accessible as
separator	`gates_separator_create`	sees a division	-	-	separator
view	`gates_view_create`, `gates_log_create`	chooses in a list, table, tree or log	arrows, PageUp/PageDown, Home, End, Enter; tree: Left/Right; typing jumps to a row; Ctrl+C copies the row; Ctrl+Left/Right pick the column F2 and Space use; with multi_select Shift/Ctrl select many, Ctrl+A all	SELECTION_CHANGED, ACTIVATED, SORT_REQUESTED, EXPAND_REQUESTED, CELL_EDITED	list, table, tree (chapter 5)
form	`gates_form_create`	fills labelled fields	-	the editors' events	group of fields (chapter 3)
property grid	`gates_propgrid_create`	edits a record's typed fields	Tab, the editors' keys	VALUE_CHANGED (result = property id)	groups of named editors (chapter 5)
dialog, menu	`gates_dialog_open`, `gates_menu_open`	answers once, picks a command	Enter, Escape, arrows	DIALOG_CLOSED, MENU_CLOSED	window, menu (chapter 4)
menu bar	`gates_menubar_create`	picks a command from a menu	F10 or Alt, arrows, letters, Escape	MENU_CLOSED, commands	menu bar (chapter 11)
toolbar	`gates_toolbar_create`	runs a command with one click	arrows, Space, Enter	commands	tool bar (chapter 11)
status bar	`gates_statusbar_create`	reads the state of the program	-	-	status bar (chapter 11)
spin box	`gates_spin_create`	types or steps a number	Up/Down, PgUp/PgDn, Enter; an arrow held repeats; the wheel when focused	VALUE_CHANGED (value)	spinner (RangeValue) (chapter 12)
slider	`gates_slider_create`	drags a number along a track	arrows, PgUp/PgDn, Home, End; the wheel when focused	VALUE_CHANGED (value)	slider (RangeValue) (chapter 12)
group box	`gates_group_create`	sees (and folds) a set of controls	Space, Enter on the title	VALUE_CHANGED (checked = expanded)	group (ExpandCollapse) (chapter 12)
image	`gates_image_create`	sees a picture	-	-	image when named (chapter 13)
tabs	`gates_tabs_create`	switches between pages	arrows, Ctrl+Tab, Ctrl+PgUp/PgDn	VALUE_CHANGED (result = index)	tab (Selection) (chapter 11)

Tab and Shift+Tab move focus through every enabled, shown control in tree order; `gates_widget_set_focusable` takes one out of the order. A disabled control (`gates_widget_set_disabled`) is drawn dimmed, ignores input and is skipped by Tab. A hidden

node (`gates_node_set_hidden`) and everything under it take no space, are not drawn and cannot be reached. Radio groups and choices name their options by stable ids (`gates_option_t`), never by position; id 0 means "nothing selected".

The older `on_click` / `on_toggle` arguments of the create functions run inside input routing and exist for compatibility; new code passes null and uses a handler.

2.2 Layout

Layout is intrinsic: every control knows the size it wants, and a container arranges its children. The program chooses the container's kind (`gates_layout_set`):

Kind	Arranges children
`GATES_LAYOUT_KIND_COLUMN`	top to bottom, each as wide as the column
`GATES_LAYOUT_KIND_ROW`	left to right, each as tall as the row
`GATES_LAYOUT_KIND_STACK`	on top of each other; only the active one shows (pages)
`GATES_LAYOUT_KIND_SPLIT`	two panes and a handle the person drags
`GATES_LAYOUT_KIND_SCROLL`	a column in a viewport, with wheel and scroll bar (a press on the track pages)
`GATES_LAYOUT_KIND_FORM`	label beside editor, row by row (chapter 3)
`GATES_LAYOUT_KIND_ABSOLUTE`	at rectangles the program gives

Padding and gap are set per container; `gates_layout_set_child_grow` lets a child take a share of the space left over, `gates_layout_set_child_align` places a child that does not stretch. Children never overlap in normal flow. A window lays its tree out before painting, in logical units; without a window, `gates_layout_run` does it.

Numbers you hand in have limits no window comes near (0.15.0): a padding or gap above `GATES_EXTENT_MAX` (65,536) is refused like a negative one, and a viewport, absolute rectangle, scroll offset or pointer position is held to +-`GATES_COORD_MAX` (16,777,216), so that the arithmetic of a layout stays inside 32 bits.

The pointer shows what a press will do (0.13.0): over a split's handle and a table's column edge it becomes the resize arrows (left-right, or up-down for stacked panes), over a text box and an editor's text the text beam, and it keeps that shape for the whole drag. A window does this by itself; `gates_cursor_at(tree, point)` answers the same question for a platform of your own.

```c
/* manual example (host): layout - a column with a growing row.
 * expect: the list takes the rest; buttons in a row */
#include <gates/gates.h>

#include <stdio.h>

#define TRY(x) do { if (!gates_is_ok(x)) return 1; } while (0)

int main(void) {
    gates_tree_t *t = nullptr;
    TRY(gates_tree_create(&(gates_tree_desc_t){0}, &t));
    gates_node_t root = gates_tree_root(t), title, list, bar, ok, cancel;
    TRY(gates_layout_set(t, root, GATES_LAYOUT_KIND_COLUMN));
    TRY(gates_layout_set_padding(t, root, 8));   /* logical units: 1/96 inch */
    TRY(gates_layout_set_gap(t, root, 6));
    TRY(gates_label_create(t, root, GATES_STR("Files"), &title));
    TRY(gates_panel_create(t, root, &list));
    TRY(gates_layout_set_child_grow(t, list, 1)); /* takes the space left over */
    TRY(gates_panel_create(t, root, &bar));
```

```
    TRY(gates_layout_set(t, bar, GATES_LAYOUT_KIND_ROW));
    TRY(gates_layout_set_gap(t, bar, 6));
    TRY(gates_button_create(t, bar, GATES_STR("OK"), nullptr, nullptr, &ok));
    TRY(gates_button_create(t, bar, GATES_STR("Cancel"), nullptr, nullptr, &cancel));
    TRY(gates_layout_run(t, (gates_size_t){ 240, 320 }, gates_text_backend_builtin()));

    gates_rect_t l = gates_node_layout_rect(t, list), a = gates_node_layout_rect(t, ok),
                 b = gates_node_layout_rect(t, cancel), r = gates_node_layout_rect(t, bar);
    /* The list ends one gap above the button row, which ends at the padding. */
    bool rest = l.y + l.h + 6 == r.y && r.y + r.h == 320 - 8;
    printf("%s; buttons %s\n", rest ? "the list takes the rest" : "the list does not grow",
           a.y == b.y && b.x > a.x ? "in a row" : "not in a row");
    gates_tree_destroy(t);
    return 0;
}
```

A scroll container follows keyboard focus: Tab into a control below the viewport and it scrolls into view. It scrolls up and down only unless you ask for more (0.10.0): `gates_layout_set_scroll_sideways(tree, area, true)` lays the children out as wide as the widest of them, and when that is wider than the viewport a bar appears along the bottom. The wheel's sideways motion (Shift+wheel on Windows) moves it, Tab brings the focused control into view across as well, and assistive technology can scroll it. `gates_layout_set_scroll_x` and `gates_layout_scroll_x` set and read the offset. Leave it off for forms: there, children as wide as the viewport are what you want. A stack page that is not active is neither drawn nor reachable.

3 Forms

In this chapter

3.1 A form holds no data 12
3.2 Draft validation 13

Header: `gates/form.h`.

3.1 A form holds no data

A form lays out labelled fields - a label beside each editor, a help line and an error line under it - and does nothing else. It does not keep values, does not validate, and does not save. The application keeps a draft, reads the editors (or follows their events), decides what is valid, puts messages on the fields, and saves after a successful submit. This split is the point: the rules about valid data belong to the program, the presentation of fields and errors to gates, and the two never argue about who holds the truth.

Each field has an id chosen by the program (`field_id`, nonzero, unique in the form); ids stay stable when rows are hidden or reordered. `gates_form_editor` gives the editor of a field.

Add	Editor	Value
`gates_form_add_text`	text box	`gates_textbox_text`
`gates_form_add_checkbox`	check box	`gates_checkbox_checked`
`gates_form_add_choice`	choice	the option id in VALUE_CHANGED
`gates_form_add_radio`	radio group	the option id in VALUE_CHANGED

`required` adds a marker to the label and tells assistive technology the field is required; `help` is a line under the editor; `gates_form_set_error` shows a message under the field in the error colour and marks a text box invalid - an empty message clears it. A narrow form puts each label above its editor instead of beside it, and is as tall as those stacked rows, so what follows it moves down rather than being covered.

```
/* manual example (host): a form, a draft, validation on submit.
 * expect: first submit: 1 error; second submit: saved Kim */
#include <gates/gates.h>

#include <stdio.h>

enum { FIELD_NAME = 1, FIELD_NEWS = 2 };

/* The application's draft: the form holds no data of its own. */
typedef struct draft_t {
    char name[32];
    bool news;
} draft_t;

/* Reads the editors, checks, and puts messages on the fields. */
static int submit(gates_tree_t *t, gates_node_t form, draft_t *d) {
    gates_str_t name = gates_textbox_text(t, gates_form_editor(t, form, FIELD_NAME));
    int errors = 0;
    if (name.size == 0) {
```

```
        (void)gates_form_set_error(t, form, FIELD_NAME, GATES_STR("enter a name"));
        errors++;
    } else {
        (void)gates_form_set_error(t, form, FIELD_NAME, GATES_STR("")); /* clears it */
        snprintf(d->name, sizeof d->name, "%.*s", (int)name.size, (const char *)name.ptr);
    }
    d->news = gates_checkbox_checked(t, gates_form_editor(t, form, FIELD_NEWS));
    return errors;
}

int main(void) {
    gates_tree_t *t = nullptr;
    if (!gates_is_ok(gates_tree_create(&(gates_tree_desc_t){0}, &t))) return 1;
    gates_node_t form, name_box;
    gates_field_desc_t name = { .label = GATES_STR("Name"), .required = true, .max_bytes = 30,
                                .help = GATES_STR("as it should appear on the badge") };
    gates_field_desc_t news = { .label = GATES_STR("News"), .text = GATES_STR("send me
news") };
    if (!gates_is_ok(gates_form_create(t, gates_tree_root(t), &form)) ||
        !gates_is_ok(gates_form_add_text(t, form, FIELD_NAME, &name, &name_box)) ||
        !gates_is_ok(gates_form_add_checkbox(t, form, FIELD_NEWS, &news, nullptr))) {
        return 1;
    }
    draft_t d = {0};
    int first = submit(t, form, &d);
    /* The error is part of the field now: screen readers read it with the name. */
    gates_access_info_t info;
    if (!gates_is_ok(gates_access_info(t, name_box, 0, &info)) || info.description.size == 0)
return 1;

    if (!gates_is_ok(gates_textbox_set_text(t, name_box, GATES_STR("Kim")))) return 1;
    int second = submit(t, form, &d);
    printf("first submit: %d error%s; second submit: %s %s\n", first, first == 1 ? "" : "s",
           second == 0 ? "saved" : "refused", d.name);
    gates_tree_destroy(t);
    return 0;
}
```

3.2 Draft validation

Validate on submit, not on every key: a person half-way through typing an e-mail address has not made a mistake yet. When a submit fails, put every error on its field, keep focus where the person can fix the first one, and say in a status line what happened (chapter 9 shows how to make that line announce itself). Clear a field's error when the person changes it, or at the next submit.

The reference application `app_settings` does all of this: required fields, help, errors, discarding changes, and saving a draft only when it is valid.

4 Commands, focus, dialogs and menus

In this chapter

4.1 One action, many ways to ask for it 14
4.2 Focus 15
4.3 Dialogs 15
4.4 Context menus 15
4.5 Undo and redo 15

Headers: `gates/command.h`, `gates/overlay.h`.

4.1 One action, many ways to ask for it

"Save" is one action whether the person clicks a button, presses Ctrl+S or picks it from a context menu. A command holds it once: an id, a label, an optional shortcut, enabled and checked states, and the function to run. Commands belong to a scope node; a shortcut works while focus is inside the scope. Buttons bound to a command (`gates_button_set_command`) take its label and enabled state, and follow it when it changes: disable the command and every way to invoke it is disabled at once.

An invocation is queued and runs at the next safe point, after the command is checked again - a command disabled in between does not run.

```
/* manual example (host): one command for a button and a shortcut.
 * expect: saved 1 time; button disabled with the command */
#include <gates/gates.h>

#include <stdio.h>

enum { CMD_SAVE = 1 };

static void on_save(gates_tree_t *tree, gates_command_id_t id, void *user) {
    (void)tree;
    (void)id;
    (*(int *)user)++;
}

int main(void) {
    gates_tree_t *t = nullptr;
    if (!gates_is_ok(gates_tree_create(&(gates_tree_desc_t){0}, &t))) return 1;
    gates_node_t root = gates_tree_root(t), save;
    int saves = 0;
    gates_command_desc_t cmd = { .id = CMD_SAVE, .label = GATES_STR("Save"), .enabled = true,
                                 .shortcut = { .letter = 'S', .ctrl = true },
                                 .invoke = on_save, .user = &saves };
    if (!gates_is_ok(gates_command_register(t, root, &cmd)) ||
        !gates_is_ok(gates_button_create(t, root, GATES_STR(""), nullptr, nullptr, &save)) ||
        !gates_is_ok(gates_button_set_command(t, save, root, CMD_SAVE))) { /* label, state from
the command */
        return 1;
    }
    /* Ctrl+S: the command is queued and runs at the next safe point. */
    gates_key_event_t ctrl_s = { .letter = 'S', .ctrl = true, .down = true };
    (void)gates_input_key(t, &ctrl_s);
```

```
    (void)gates_tree_dispatch_events(t, 0);

    /* Disabling the command disables every button bound to it, and its shortcut. */
    if (!gates_is_ok(gates_command_set_enabled(t, root, CMD_SAVE, false))) return 1;
    (void)gates_input_key(t, &ctrl_s);
    (void)gates_tree_dispatch_events(t, 0);

    gates_access_info_t info;
    if (!gates_is_ok(gates_access_info(t, save, 0, &info))) return 1;
    printf("saved %d time%s; button %s with the command\n", saves, saves == 1 ? "" : "s",
           (info.states & GATES_ACCESS_DISABLED) != 0 ? "disabled" : "enabled");
    gates_tree_destroy(t);
    return 0;
}
```

A command may have a role: the scope's DEFAULT command runs when Enter is pressed in a text box or check box of the scope, its CANCEL command on Escape anywhere in the scope (for a dialog: OK and Cancel).

4.2 Focus

One control has keyboard focus (`gates_tree_focus`, `gates_tree_set_focus`). Tab and Shift+Tab move it in tree order and wrap inside the current focus scope; the window's scope is the whole tree, an open dialog's scope is the dialog. A focused control that is hidden or destroyed lets focus go (a press or drag inside it is let go too); the program decides where focus goes next with `gates_tree_set_focus`. When a dialog closes, focus returns to the control that had it.

4.3 Dialogs

`gates_dialog_open` shows a modal dialog inside the window: the rest of the window is dimmed and ignores input, Tab stays inside, and the window's own shortcuts wait. Opening returns at once - there is no nested event loop. The program fills the dialog's content panel, registers the dialog's commands with the dialog as scope, and learns the answer from one DIALOG_CLOSED event (`ev->result`: ACCEPTED or CANCELED). Focus returns to where it was. At most eight dialogs and menus are open at a time.

4.4 Context menus

`gates_menu_open` lists commands of a scope at a point: their labels, check marks, shortcuts and enabled states come from the commands. Choosing an entry invokes its command through the same queued, checked path as a shortcut. Arrows move, Enter chooses, Escape or a click outside only closes (a click outside never also clicks what is under it).

`gates_command_set_submenu(tree, scope, id, ids, count)` gives an entry a submenu of other commands of the same scope. The entry shows an arrow; Right, Enter, a click, or resting the pointer on it opens the submenu beside it, and Left or Escape closes it again. Choosing inside a submenu invokes that command and closes every menu of the chain; the menu you opened reports MENU_CLOSED with the command's id. The same works in menu bar menus.

4.5 Undo and redo

Text boxes undo their own typing. For the program's own data - a renamed item, a moved row, a changed setting - `gates_undo_create` (gates/undo.h) gives a stack: after making a change, push an entry with a label and two functions that take it back and make it again. Entries with the same merge key in a row become one (typing into one field), the stack keeps a bounded number, and `gates_undo_mark_clean` / `gates_undo_is_clean` tell whether the data is as it was saved.

`gates_undo_bind` keeps two commands in step: enabled only when there is something to undo or redo, and labelled "Undo Rename" / "Redo Rename" (with words in the program's language).

```c
/* manual example (host): Undo and Redo commands over the program's own data.
 * expect: menu shows "Undo Rename"; after Undo: name Draft, menu shows "Redo Rename"; clean
again: yes */
#include <gates/gates.h>

#include <stdio.h>
#include <string.h>

enum { CMD_UNDO = 1, CMD_REDO = 2 };

static char name[32] = "Draft";

/* One entry remembers the name before and after the change. */
typedef struct rename_t {
    char before[32], after[32];
} rename_t;

static gates_err_t undo_rename(void *data) {
    memcpy(name, ((rename_t *)data)->before, sizeof name);
    return GATES_OK;
}
static gates_err_t redo_rename(void *data) {
    memcpy(name, ((rename_t *)data)->after, sizeof name);
    return GATES_OK;
}

static void on_command(gates_tree_t *tree, gates_command_id_t id, void *user) {
    (void)tree;
    gates_undo_t *undo = user;
    (void)(id == CMD_UNDO ? gates_undo_undo(undo) : gates_undo_redo(undo));
}

static void print_label(gates_tree_t *t, gates_command_id_t id) {
    gates_str_t l = gates_command_label(t, gates_tree_root(t), id);
    printf("\"%.*s\"", (int)l.size, (const char *)l.ptr);
}

int main(void) {
    gates_tree_t *t = nullptr;
    gates_undo_t *undo = nullptr;
    if (!gates_is_ok(gates_tree_create(&(gates_tree_desc_t){0}, &t)) ||
        !gates_is_ok(gates_undo_create((gates_allocator_t){0}, 0, &undo))) {
        return 1;
    }
    gates_node_t root = gates_tree_root(t);
    gates_command_desc_t cmds[] = {
        { .id = CMD_UNDO, .label = GATES_STR_INIT("Undo"), .shortcut = { .key =
GATES_KEY_Z, .ctrl = true },
          .enabled = true, .invoke = on_command, .user = undo },
        { .id = CMD_REDO, .label = GATES_STR_INIT("Redo"), .shortcut = { .key =
GATES_KEY_Y, .ctrl = true },
          .enabled = true, .invoke = on_command, .user = undo },
    };
    if (!gates_is_ok(gates_command_register(t, root, &cmds[0])) || !
gates_is_ok(gates_command_register(t, root, &cmds[1])) ||
        !gates_is_ok(gates_undo_bind(undo, t, root, CMD_UNDO, CMD_REDO, GATES_STR("Undo"),
GATES_STR("Redo")))) {
        return 1;
```

```
    }
    gates_undo_mark_clean(undo); /* the name as saved */

    /* The program renames, then records how to take it back. */
    static rename_t change = { "Draft", "Final" };
    memcpy(name, change.after, sizeof name);
    gates_undo_entry_t entry = { .label = GATES_STR_INIT("Rename"), .undo =
undo_rename, .undo_data = &change,
                                 .redo = redo_rename, .redo_data = &change };
    if (!gates_is_ok(gates_undo_push(undo, &entry))) return 1;
    printf("menu shows ");
    print_label(t, CMD_UNDO);

    /* A person presses Ctrl+Z. */
    gates_key_event_t ctrl_z = { .key = GATES_KEY_Z, .ctrl = true, .down = true };
    (void)gates_input_key(t, &ctrl_z);
    (void)gates_tree_dispatch_events(t, 0);
    printf("; after Undo: name %s, menu shows ", name);
    print_label(t, CMD_REDO);
    printf("; clean again: %s\n", gates_undo_is_clean(undo) ? "yes" : "no");

    gates_undo_destroy(undo);
    gates_tree_destroy(t);
    return 0;
}
```

5 Views over your data

In this chapter

5.1 One node for any number of rows 18
5.2 Lists, tables, trees and logs 20
5.3 Cells people can change 20
5.4 Many rows at once: multi-selection 23
5.5 A record's fields: the property grid 25
5.6 Rules for models 26

Header: `gates/view.h`.

5.1 One node for any number of rows

A list with a million rows is still one node. The rows belong to the application: it describes them through a model - how many there are, the id at a position, the position of an id, the text of a cell - and gates asks only for the rows it shows (at most `GATES_VIEW_MAX_ROWS` at a time). Nothing is copied into gates, and scrolling through a million rows costs what scrolling through a screenful costs.

Rows have ids (`gates_item_id_t`, nonzero, stable): the selection is an id, not a position, so it stays on the same row when rows are inserted above it, removed elsewhere, or sorted. After the application changes its rows it calls `gates_view_model_changed`; when the selected row is gone, the selection moves to the nearest remaining row and a SELECTION_CHANGED with origin PROGRAM says so.

```
/* manual example (host): a list over the application's own rows.
 * expect: selected 3 (Cherry); after removing it: 4 (Damson), origin program */
#include <gates/gates.h>

#include <stdio.h>
#include <string.h>

typedef struct row_t {
    gates_item_id_t id;          /* stable: survives inserts, removals, sorting */
    const char *name;
} row_t;

typedef struct fruits_t {
    row_t rows[8];
    gates_u64 n;
} fruits_t;

/* The model: gates asks only for the rows it shows, by position and by id. */
static gates_u64 f_count(void *u) { return ((fruits_t *)u)->n; }
static gates_item_id_t f_id_at(void *u, gates_u64 row) { return ((fruits_t *)u)-
>rows[row].id; }
static bool f_index_of(void *u, gates_item_id_t id, gates_u64 *row) {
    fruits_t *f = u;
    for (gates_u64 i = 0; i < f->n; i++) {
        if (f->rows[i].id == id) { *row = i; return true; }
    }
    return false;
}
static gates_err_t f_cell(void *u, gates_item_id_t id, gates_column_id_t col, gates_cell_t
```

```
*out) {
    (void)col;
    gates_u64 row;
    if (!f_index_of(u, id, &row)) return PROVEN_ERR_INVALID_ARG;
    const char *s = ((fruits_t *)u)->rows[row].name;
    out->text = (gates_str_t){ .ptr = (const gates_u8 *)s, .size = strlen(s) };
    return GATES_OK;
}

static gates_item_id_t last_id;
static gates_event_origin_t last_origin;

static void on_view(gates_tree_t *tree, const gates_event_t *ev, void *user) {
    (void)tree;
    (void)user;
    if (ev->kind == GATES_EVENT_SELECTION_CHANGED) {
        last_id = ev->item;
        last_origin = ev->origin;
    }
}

static const char *name_of(fruits_t *f, gates_item_id_t id) {
    gates_u64 row;
    return f_index_of(f, id, &row) ? f->rows[row].name : "none";
}

int main(void) {
    fruits_t f = { .rows = { { 1, "Apple" }, { 2, "Banana" }, { 3, "Cherry" }, { 4, "Damson" },
{ 5, "Elder" } }, .n = 5 };
    gates_tree_t *t = nullptr;
    if (!gates_is_ok(gates_tree_create(&(gates_tree_desc_t){0}, &t))) return 1;
    gates_node_t list;
    gates_rows_model_t model = { .user = &f, .count = f_count, .id_at = f_id_at, .index_of =
f_index_of, .cell = f_cell };
    if (!gates_is_ok(gates_view_create(t, gates_tree_root(t), &(gates_view_desc_t){0}, &list))
||
        !gates_is_ok(gates_view_set_model(t, list, &model)) ||
        !gates_is_ok(gates_widget_set_handler(t, list, on_view, nullptr)) ||
        !gates_is_ok(gates_layout_run(t, (gates_size_t){ 200, 160 },
gates_text_backend_builtin()))) {
        return 1;
    }
    /* A person picks the third row. */
    if (!gates_is_ok(gates_access_select(t, list, 3))) return 1;
    (void)gates_tree_dispatch_events(t, 0);
    printf("selected %llu (%s); ", (unsigned long long)last_id, name_of(&f, last_id));

    /* The application removes that row, then tells the view its rows changed:
     * the selection moves to the nearest remaining row and says so. */
    memmove(&f.rows[2], &f.rows[3], 2 * sizeof f.rows[0]);
    f.n = 4;
    if (!gates_is_ok(gates_view_model_changed(t, list))) return 1;
    (void)gates_tree_dispatch_events(t, 0);
    printf("after removing it: %llu (%s), origin %s\n", (unsigned long long)last_id,
name_of(&f, last_id),
           last_origin == GATES_ORIGIN_PROGRAM ? "program" : "user");
    gates_tree_destroy(t);
    return 0;
}
```

5.2 Lists, tables, trees and logs

- A list has no columns; the cell asked for is column 0.
- A table has columns (`gates_column_desc_t`: id, label, width, minimum) and may show a header. Clicking a header cell sends SORT_REQUESTED with the column id: the application sorts its rows and calls `gates_view_model_changed`. Dragging a header edge resizes a column. `gates_view_set_column_hidden` and `gates_view_move_column` choose and order the columns; with `column_menu` in the description a right press on the header (or Shift+F10 on the table) opens a menu with a checked entry per column, and the last shown column cannot be hidden. gates/state.h saves widths, order and hidden columns.
- A tree is a list whose model also answers `row_info`: depth, whether the row can open, whether it is open, a loading or error state. Opening and closing are requests (EXPAND_REQUESTED): the application changes its flattened rows and calls `gates_view_model_changed`. gates never walks rows it does not show.
- A log (`gates_log_create`) owns its lines: append text, keep a line and byte limit, drop the oldest, and follow the end while the person has not scrolled away. Scrolling away or back reports GATES_EVENT_FOLLOW_CHANGED (`ev->result` 1 while following), so a status line can say so without watching the pointer.

Every view also answers typing: the letters typed select the next row whose cell in the current column - the one outlined, moved with Ctrl+Left/Right or a click - starts with them, or its first shown cell when there is no current column (0.10.0; a pause of a second starts over; the same letter again steps through such rows). Ctrl+C puts the selected row's shown cells on the clipboard, separated by tabs.

5.3 Cells people can change

A column can show more than text (`gates_column_desc_t.kind`): `GATES_CELL_CHECK` draws a check box from `cell.checked`, `GATES_CELL_PROGRESS` a bar from `cell.permille` (0 to 1000), and `GATES_CELL_ICON_TEXT` a 16 x 16 icon (`cell.icon`, see chapter 13) before the text. A column with a `paint` function draws its cells itself, clipped to each cell.

A column marked `editable` lets a person change the selected row's cell, and the model says yes or no through its `set_cell` function. F2 or a double click opens a text box over the cell with the model's text selected; Enter commits, Escape cancels, and moving focus away commits. Space, or a click on the box, toggles a check column. Which column F2 and Space use follows the table's current column - the cell last pressed, or moved with Ctrl+Left and Ctrl+Right, outlined in the selected row - when it is an editable column of the right kind, else the first such column. When `set_cell` returns an error the editor stays open and is marked invalid (after focus has left, the edit is dropped instead). After a change the view reads the model again and sends CELL_EDITED with the row (`ev->item`) and the column (`ev->result`). The program can do the same with `gates_view_edit` and `gates_view_end_edit`.

Screen readers see every cell of a table row as an element of its own: its text, its column's label, a check cell's state and a progress cell's percentage. They can toggle a check cell or set a text cell's value when the column is editable - through `set_cell`, reported with CELL_EDITED, refused exactly as a person's edit would be (`gates_access_cell_info`, `_toggle`, `_set_value`). A table wider than its view scrolls sideways for them too.

```
/* manual example (host): a shopping list edited in place - a text column and a check column.
 * expect: item 2 is now Bread (column 1), done: yes (column 2); refused empty name: editor
still open */
#include <gates/gates.h>

#include <stdio.h>
#include <string.h>
```

```
enum { COL_NAME = 1, COL_DONE = 2 };

typedef struct item_t {
    gates_item_id_t id;
    char name[32];
    bool done;
} item_t;

typedef struct list_t {
    item_t items[4];
    gates_u64 n;
} list_t;

static gates_u64 l_count(void *u) { return ((list_t *)u)->n; }
static gates_item_id_t l_id_at(void *u, gates_u64 row) { return ((list_t *)u)->items[row].id; }
static bool l_index_of(void *u, gates_item_id_t id, gates_u64 *row) {
    list_t *l = u;
    for (gates_u64 i = 0; i < l->n; i++) {
        if (l->items[i].id == id) { *row = i; return true; }
    }
    return false;
}
static gates_err_t l_cell(void *u, gates_item_id_t id, gates_column_id_t col, gates_cell_t
*out) {
    gates_u64 row;
    if (!l_index_of(u, id, &row)) return PROVEN_ERR_INVALID_ARG;
    item_t *it = &((list_t *)u)->items[row];
    if (col == COL_DONE) {
        out->checked = it->done;
    } else {
        out->text = (gates_str_t){ .ptr = (const gates_u8 *)it->name, .size = strlen(it-
>name) };
    }
    return GATES_OK;
}
/* A person changed a cell: the model decides. An empty name is refused. */
static gates_err_t l_set_cell(void *u, gates_item_id_t id, gates_column_id_t col, const
gates_cell_t *value) {
    gates_u64 row;
    if (!l_index_of(u, id, &row)) return PROVEN_ERR_INVALID_ARG;
    item_t *it = &((list_t *)u)->items[row];
    if (col == COL_DONE) {
        it->done = value->checked;
        return GATES_OK;
    }
    if (value->text.size == 0 || value->text.size >= sizeof it->name) return
PROVEN_ERR_INVALID_ARG;
    memcpy(it->name, value->text.ptr, value->text.size);
    it->name[value->text.size] = 0;
    return GATES_OK;
}

static list_t list = { .items = { { 1, "Milk", false }, { 2, "Bred", false }, { 3, "Eggs",
true } }, .n = 3 };

static void on_view(gates_tree_t *tree, const gates_event_t *ev, void *user) {
    (void)tree;
    (void)user;
    if (ev->kind != GATES_EVENT_CELL_EDITED) return;
    gates_u64 row;
```

```
    if (!l_index_of(&list, ev->item, &row)) return;
    if (ev->result == COL_NAME) {
        printf("item %llu is now %s (column %u), ", (unsigned long long)ev->item,
list.items[row].name, ev->result);
    } else {
        printf("done: %s (column %u); ", list.items[row].done ? "yes" : "no", ev->result);
    }
}

static void press(gates_tree_t *t, gates_key_t key) {
    gates_key_event_t ev = { .key = key, .down = true };
    (void)gates_input_key(t, &ev);
    ev.down = false;
    (void)gates_input_key(t, &ev);
}

static void type(gates_tree_t *t, const char *s) {
    for (; *s != 0; s++) (void)gates_input_char(t, (gates_u32)(unsigned char)*s);
}

int main(void) {
    gates_tree_t *t = nullptr;
    if (!gates_is_ok(gates_tree_create(&(gates_tree_desc_t){0}, &t))) return 1;
    const gates_column_desc_t cols[] = {
        { .id = COL_NAME, .label = GATES_STR_INIT("Item"), .width = 120, .editable = true },
        { .id = COL_DONE, .label = GATES_STR_INIT("Done"), .width = 60, .kind =
GATES_CELL_CHECK, .editable = true },
    };
    gates_rows_model_t model = { .user = &list, .count = l_count, .id_at = l_id_at, .index_of =
l_index_of,
                                 .cell = l_cell, .set_cell = l_set_cell };
    gates_node_t table;
    if (!gates_is_ok(gates_view_create(t, gates_tree_root(t),
                                       &(gates_view_desc_t){ .columns = cols, .column_count =
2, .header = true },
                                       &table)) ||
        !gates_is_ok(gates_view_set_model(t, table, &model)) ||
        !gates_is_ok(gates_widget_set_handler(t, table, on_view, nullptr)) ||
        !gates_is_ok(gates_layout_run(t, (gates_size_t){ 240, 160 },
gates_text_backend_builtin()))) {
        return 1;
    }
    /* A person selects the second row, presses F2, retypes the name and presses Enter. */
    gates_tree_set_focus(t, table);
    press(t, GATES_KEY_DOWN);
    press(t, GATES_KEY_DOWN);
    press(t, GATES_KEY_F2);
    type(t, "Bread");
    press(t, GATES_KEY_ENTER);
    /* Space toggles the row's check column. */
    press(t, GATES_KEY_SPACE);
    (void)gates_tree_dispatch_events(t, 0);

    /* An empty name is refused: the editor stays open, marked invalid, until Escape. */
    press(t, GATES_KEY_F2);
    press(t, GATES_KEY_DELETE);
    press(t, GATES_KEY_ENTER);
    printf("refused empty name: editor %s\n", gates_view_editing(t, table, nullptr, nullptr) ?
"still open" : "closed");
    press(t, GATES_KEY_ESCAPE);
    gates_tree_destroy(t);
```

```
    return 0;
}
```

5.4 Many rows at once: multi-selection

A view made with `multi_select` lets a person select many rows: Shift+click and Shift with the arrows make a range from an anchor, Ctrl+click and Ctrl+Space add or remove one row, Ctrl+A takes all, and Ctrl with the arrows moves the focus without changing the selection. Pressing on a row and dragging selects the rows passed over (with Ctrl held at the press, adds them); past the top or bottom of the rows the view scrolls one row for each pointer move (0.10.0). The selection is the program's, like the rows: the model answers `next_selected(row)` - the first selected row at or after `row` - and each gesture arrives as GATES_EVENT_SELECT_REQUESTED (`ev->result` says ONE, TOGGLE, RANGE, ADD_RANGE or ALL; `ev->item` is the target row and `ev->anchor` where the range starts). The program changes its selection and calls `gates_view_model_changed`.

Because gates only asks about rows it paints, describes or copies, the program can keep any number of selected rows in a small form - a few ranges; "all" of a million rows is one range. gates has such a store (0.10.0): `gates_selection_create` makes one, the model's `next_selected` answers `gates_selection_next`, and `gates_view_apply_selection(tree, ev, sel)` applies a request (ids turned into rows through the model) and repaints. When rows are inserted or removed, `gates_selection_rows_inserted` / `_rows_removed` move the ranges with them; `gates_selection_range` walks them for the program's own use. A program with its own idea of a selection keeps it instead - the view never reads the store. Ctrl+C copies up to `GATES_VIEW_COPY_MAX` (10000) selected rows itself, one per line; past that it sends GATES_EVENT_COPY_REQUESTED and leaves the copying to the program. `gates_view_selected` stays the focus row. Screen readers see a list that selects many (`gates_access_set_item_selected` joins or leaves a row, as a request).

```
/* manual example (host): a million rows, selected together - kept in a selection store.
 * expect: 0..999999 after Shift+End; 1..999999 after Ctrl+Space on row 0; 0..999999 after
Ctrl+A; copy asked for 1000000 rows */
#include <gates/gates.h>

#include <stdio.h>

#define ROWS 1000000u

/* The model: row r has id r + 1; the store answers next_selected. */
typedef struct app_t {
    gates_selection_t *sel;
    gates_node_t view;
    int copy_asked;
} app_t;

static gates_u64 m_count(void *u) { (void)u; return ROWS; }
static gates_item_id_t m_id_at(void *u, gates_u64 row) { (void)u; return row < ROWS ? row + 1 :
0; }
static bool m_index_of(void *u, gates_item_id_t id, gates_u64 *row) {
    (void)u;
    if (id == 0 || id > ROWS) return false;
    *row = id - 1;
    return true;
}
static gates_err_t m_cell(void *u, gates_item_id_t id, gates_column_id_t col, gates_cell_t
*out) {
    (void)u; (void)id; (void)col;
    out->text = GATES_STR("a row");
```

```
    return GATES_OK;
}
static gates_u64 m_next_selected(void *u, gates_u64 row) { return gates_selection_next(((app_t
*)u)->sel, row); }

/* A person's gesture is a request: the store applies it, then the view repaints. */
static void on_view(gates_tree_t *t, const gates_event_t *ev, void *user) {
    app_t *a = user;
    if (ev->kind == GATES_EVENT_COPY_REQUESTED) a->copy_asked++;
    if (ev->kind == GATES_EVENT_SELECT_REQUESTED) (void)gates_view_apply_selection(t, ev, a-
>sel);
}

static void press(gates_tree_t *t, gates_key_t key, bool ctrl, bool shift) {
    gates_key_event_t ev = { .key = key, .ctrl = ctrl, .shift = shift, .down = true };
    (void)gates_input_key(t, &ev);
    (void)gates_tree_dispatch_events(t, 0);
}

static void show(const app_t *a, const char *after, bool last) {
    gates_u64 lo = 0, hi = 0;
    for (gates_u32 i = 0; gates_selection_range(a->sel, i, &lo, &hi); i++) {
        printf("%s%llu..%llu", i > 0 ? "," : "", (unsigned long long)lo, (unsigned long
long)hi);
    }
    printf(" after %s%s", after, last ? "" : "; ");
}

static gates_err_t no_clip_get(void *ctx, gates_allocator_t al, gates_u8 **out, gates_usize_t
*n) {
    (void)ctx; (void)al;
    *out = nullptr;
    *n = 0;
    return GATES_OK;
}
static gates_err_t no_clip_set(void *ctx, gates_str_t text) { (void)ctx; (void)text; return
GATES_OK; }

int main(void) {
    gates_tree_t *t = nullptr;
    static app_t a;
    if (!gates_is_ok(gates_tree_create(&(gates_tree_desc_t){0}, &t)) ||
        !gates_is_ok(gates_selection_create((gates_allocator_t){0}, &a.sel))) {
        return 1;
    }
    gates_rows_model_t model = { .user = &a, .count = m_count, .id_at = m_id_at, .index_of =
m_index_of,
                                 .cell = m_cell, .next_selected = m_next_selected };
    if (!gates_is_ok(gates_view_create(t, gates_tree_root(t), &(gates_view_desc_t)
{ .multi_select = true }, &a.view)) ||
        !gates_is_ok(gates_view_set_model(t, a.view, &model)) ||
        !gates_is_ok(gates_widget_set_handler(t, a.view, on_view, &a)) ||
        !gates_is_ok(gates_layout_run(t, (gates_size_t){ 300, 200 },
gates_text_backend_builtin()))) {
        return 1;
    }
    gates_tree_set_clipboard(t, &(gates_clipboard_t){ .get_text = no_clip_get, .set_text =
no_clip_set });
    gates_tree_set_focus(t, a.view);
    press(t, GATES_KEY_DOWN, false, false);  /* row 0 alone */
    press(t, GATES_KEY_END, false, true);    /* Shift+End: 0 to the last row */
```

```
    show(&a, "Shift+End", false);
    press(t, GATES_KEY_HOME, true, false);   /* Ctrl+Home: the focus moves, nothing asked */
    press(t, GATES_KEY_SPACE, true, false);  /* Ctrl+Space: row 0 leaves the selection */
    show(&a, "Ctrl+Space on row 0", false);
    press(t, GATES_KEY_A, true, false);      /* Ctrl+A: all, one range */
    show(&a, "Ctrl+A", false);
    press(t, GATES_KEY_C, true, false);      /* more than GATES_VIEW_COPY_MAX rows: the program
is asked */
    printf("copy asked for %u rows\n", a.copy_asked == 1 ? ROWS : 0u);
    gates_tree_destroy(t);
    gates_selection_destroy(a.sel);
    return 0;
}
```

5.5 A record's fields: the property grid

When one record has many typed fields - a document's settings, the selected object in an inspector - `gates_propgrid_create` (gates/propgrid.h) lays them out as rows of a name and an editor: `gates_propgrid_add_text`, `_bool`, `_choice` and `_number` make a text box, check box, choice or spin box. Properties are grouped by category, each a collapsible group box; properties without one come first. Every property has a stable id: `gates_propgrid_editor` gives its editor for setting and reading the value with the editor's own functions, and one handler (`gates_propgrid_set_handler`) hears every change a person makes as VALUE_CHANGED with the property id in `ev->result` and the value in `ev->text`, `ev->checked` or `ev->value`.

```
/* manual example (host): a property grid over a document's settings.
 * expect: 4 properties in 2 categories; property 2 (Pages) is now 13; property 3 (Draft) is
now off */
#include <gates/gates.h>

#include <stdio.h>

enum { P_TITLE = 1, P_PAGES, P_DRAFT, P_PAPER };

/* One handler hears every change, by property id. */
static void on_property(gates_tree_t *tree, const gates_event_t *ev, void *user) {
    (void)tree;
    (void)user;
    if (ev->result == P_PAGES) printf("property %u (Pages) is now %lld; ", ev->result, (long
long)ev->value);
    if (ev->result == P_DRAFT) printf("property %u (Draft) is now %s\n", ev->result, ev-
>checked ? "on" : "off");
}

int main(void) {
    gates_tree_t *t = nullptr;
    if (!gates_is_ok(gates_tree_create(&(gates_tree_desc_t){0}, &t))) return 1;
    gates_node_t grid;
    const gates_option_t papers[] = { { .id = 1, .label = GATES_STR_INIT("A4") }, { .id =
2, .label = GATES_STR_INIT("Letter") } };
    gates_range_t pages = { .min = 1, .max = 999, .value = 12 };
    if (!gates_is_ok(gates_propgrid_create(t, gates_tree_root(t), &grid)) ||
        !gates_is_ok(gates_propgrid_add_text(t, grid, GATES_STR("Document"), P_TITLE,
GATES_STR("Title"), GATES_STR("Notes"))) ||
        !gates_is_ok(gates_propgrid_add_number(t, grid, GATES_STR("Document"), P_PAGES,
GATES_STR("Pages"), &pages)) ||
        !gates_is_ok(gates_propgrid_add_bool(t, grid, GATES_STR("Print"), P_DRAFT,
GATES_STR("Draft"), true)) ||
```

```
        !gates_is_ok(gates_propgrid_add_choice(t, grid, GATES_STR("Print"), P_PAPER,
GATES_STR("Paper"), papers, 2, 1)) ||
        !gates_is_ok(gates_propgrid_set_handler(t, grid, on_property, nullptr)) ||
        !gates_is_ok(gates_layout_run(t, (gates_size_t){ 320, 320 },
gates_text_backend_builtin()))) {
        return 1;
    }
    printf("%u properties in %d categories; ", gates_propgrid_count(t, grid),
           !gates_node_eq(gates_propgrid_category(t, grid, GATES_STR("Print")),
GATES_NODE_NULL) ? 2 : 1);

    /* A person steps Pages up in its spin box, then turns Draft off with Space. */
    gates_tree_set_focus(t, gates_node_first_child(t, gates_propgrid_editor(t, grid,
P_PAGES)));
    gates_key_event_t up = { .key = GATES_KEY_UP, .down = true };
    (void)gates_input_key(t, &up);
    gates_tree_set_focus(t, gates_propgrid_editor(t, grid, P_DRAFT));
    gates_key_event_t space = { .key = GATES_KEY_SPACE, .down = true };
    (void)gates_input_key(t, &space);
    space.down = false;
    (void)gates_input_key(t, &space);
    (void)gates_tree_dispatch_events(t, 0);
    gates_tree_destroy(t);
    return 0;
}
```

5.6 Rules for models

Model callbacks run on the UI thread, never during event dispatch or inside another callback, and never after `gates_view_set_model(view, nullptr)` returns. A cell's text is borrowed until the next call into the model. The model must not change between `gates_view_model_changed` calls without telling the view; `set_cell` is the one callback that may change it (the view reads it again afterwards).

6 Text entry

In this chapter

6.1 The text box 27
6.2 The limit is a question 27
6.3 The installed input method 28
6.4 Unicode limits in 0.11.0 28
6.5 The clipboard 29
6.6 Other encodings at the edge 29
6.7 Many lines: the editor 31
6.8 Large texts 33

Headers: `gates/widget.h` (text box), `gates/text_edit.h`, `gates/clipboard.h`, `gates/input.h`, `gates/ui.h`.

6.1 The text box

A text box edits one line of UTF-8 text. The person types, selects with Shift and the pointer (Shift+click extends, a double click takes a word, a triple click all of it), moves by character and to the start or end (Home, End), copies, cuts and pastes (Ctrl+C, Ctrl+X, Ctrl+V), and undoes and redoes (Ctrl+Z, Ctrl+Y). Each successful edit queues TEXT_CHANGED with the committed text; the program's own `gates_textbox_set_text` is silent and clears the undo history. Under every text box is the edit core (`gates/text_edit.h`): text, caret, selection and an open composition, all as byte offsets into UTF-8.

Setting	Effect
`gates_textbox_set_read_only`	focus, selection and copy work; editing, paste, cut, undo and composition are refused
`gates_textbox_set_password`	one * per character, no copy or cut, events carry no text, no undo, no IME
`gates_textbox_set_max_bytes`	a maximum length in UTF-8 bytes (see below)
`gates_textbox_set_undo_limits`	undo history bounds; defaults 64 entries, 16384 bytes
`gates_textbox_set_invalid`	the error look (a form sets it for you)

6.2 The limit is a question

When input would pass the maximum length, gates inserts nothing and asks: a LIMIT_EXCEEDED event carries the refused input and how many of its bytes would fit. The program puts the question to the person - keep what fits, or drop it - and answers with `gates_textbox_accept_fit` or `gates_textbox_discard_rejected`. Silently cutting a pasted account number in half is the failure this avoids.

```
/* manual example (host): a text box with a limit - the overflow is a question.
 * expect: offered 7 bytes, 5 fit; kept "Seoul" */
#include <gates/gates.h>

#include <stdio.h>

static gates_u32 offered, fit;
```

```
static void on_box(gates_tree_t *tree, const gates_event_t *ev, void *user) {
    (void)tree;
    (void)user;
    if (ev->kind == GATES_EVENT_LIMIT_EXCEEDED) {
        offered = (gates_u32)ev->text.size;
        fit = ev->fit_bytes;
    }
}

int main(void) {
    gates_tree_t *t = nullptr;
    if (!gates_is_ok(gates_tree_create(&(gates_tree_desc_t){0}, &t))) return 1;
    gates_node_t city;
    if (!gates_is_ok(gates_textbox_create(t, gates_tree_root(t), GATES_STR(""), 20, &city)) ||
        !gates_is_ok(gates_textbox_set_max_bytes(t, city, 5)) ||
        !gates_is_ok(gates_widget_set_handler(t, city, on_box, nullptr))) {
        return 1;
    }
    gates_tree_set_focus(t, city);
    /* Typed or pasted input (an IME delivers its result the same way). */
    (void)gates_input_commit(t, GATES_STR("Seoul!!"));
    (void)gates_tree_dispatch_events(t, 0);
    /* Nothing was inserted: the box holds the input as an offer. Here the
     * answer is "keep what fits"; gates_textbox_discard_rejected drops it. */
    if (!gates_is_ok(gates_textbox_accept_fit(t, city))) return 1;
    gates_str_t text = gates_textbox_text(t, city);
    printf("offered %u bytes, %u fit; kept \"%.*s\"\n", offered, fit, (int)text.size, (const
char *)text.ptr);
    gates_tree_destroy(t);
    return 0;
}
```

6.3 The installed input method

gates implements no input method. On Windows the installed IME (Korean, Japanese, Chinese, ...) works in every text box through IMM32: the syllable being composed is shown inline at the caret, the candidate window opens under the caret, and the finished text arrives as one edit. A read-only or password box turns the IME off while it has focus. For a program that feeds text itself - tests, automation - `gates_input_commit` delivers text as an IME would, and `gates_input_preedit` shows a composition.

6.4 Unicode limits in 0.11.0

- Text is UTF-8 everywhere; invalid input is refused, never repaired silently. Text in other encodings is converted at the edge (below).
- Every character has its own advance, taken from the font: text is proportional in the UI font and fixed-pitch in the mono font (chapter 8). A string is exactly as wide as the sum of its characters, so the caret, a selection and a click always land between characters.
- There is no kerning, no ligatures and no shaping: scripts that need shaping (Arabic, the Indic scripts, Thai) show their characters one by one, and right-to-left text is not reordered.
- Editing moves by code point, not by grapheme cluster: a base letter and a combining mark are two steps.
- The Win32 backend draws real system glyphs, taking characters its face lacks (Hangul in Segoe UI) from the system's fallback fonts; the builtin backend (tests, headless) is fixed-pitch and draws ASCII, with a box of the right width for everything else.
- A text box is one line; many lines are the editor's (below).

6.5 The clipboard

The window provides the platform clipboard (`gates/clipboard.h`): text crosses it as UTF-8 and is converted at the edge. On Windows a line end is CR LF on the clipboard: copy writes LF as CR LF and paste reads CR LF as LF, so a text keeps its line ends through other programs (0.15.1). A tree without a clipboard provider (headless) makes copy, cut and paste do nothing.

6.6 Other encodings at the edge

Inside a program everything is UTF-8 - the API, the text box, the editor, the text buffer. The outside is not always: a file may be UTF-16 with a byte order mark, a console on Korean Windows uses code page 949, an older program writes EUC-KR. `gates/encoding.h` converts at the edge, so the program keeps one encoding inside and meets each environment in its own (0.10.0):

- `gates_encoding_to_utf8(enc, bytes, size, flags, alloc, &out, &out_size, &bad_at)` brings text in; a byte order mark of that encoding is skipped. `gates_encoding_from_utf8` takes it out again (with a mark first when `enc.bom`). Results come from `alloc` ({0} = the heap), end with zero bytes that are not counted (a C string, or a wide string, as it is), and are freed by the caller.
- UTF-8, UTF-16 and UTF-32 in both byte orders are done by gates everywhere. Code pages go by their Windows numbers (`GATES_CODEPAGE_CP949` 949, `GATES_CODEPAGE_EUC_KR` 51949, 932, 936, 1252, ...; 0 = the system's) and need the platform's tables: the Win32 app installs a converter when it is created; without one they are UNSUPPORTED. `gates_encoding_system_codepage` and `gates_encoding_console_codepage` say what the system and the console use.
- Broken input becomes U+FFFD; with `GATES_ENCODING_STRICT` the call refuses it instead and `bad_at` says which byte. A character a code page cannot hold becomes its `?`, or a refusal when strict.
- `gates_encoding_detect` guesses for bytes of unknown origin: a byte order mark decides, then UTF-16 by its zero bytes, then valid UTF-8, else the system code page. `gates_utf8_valid` checks text before it goes into the editor (which refuses anything else).
- With no OS to ask (a microcontroller, an RTOS GUI), `gates_codepage_converter_cp949()` is a converter of gates' own for 949 (CP949) and 51949 (EUC-KR), from tables measured on Windows 11, so it converts as Windows does (0.11.0). Install it with `gates_encoding_set_codepage_converter`; only a program that calls it carries its tables (about 120 KB).
- Without allocating, for targets that count their memory: `gates_encoding_to_utf8_buf` / `_from_utf8_buf` write into your buffer (a call with none asks for the size), and a `gates_encoding_stream_t` converts in pieces - a file read in blocks, bytes from a serial line - carrying a character cut at a piece's end over to the next. Both take the UTF forms.
- On Windows, `main`'s arguments are in the ANSI code page and `printf` of UTF-8 shows garbled on a console that is not set to UTF-8: `gates_args_utf8_win32` gives the command line as UTF-8 arguments (split by `gates_args_split`, the Windows rules, usable anywhere), and `gates_console_write_win32` writes UTF-8 to the console, or the bytes as they are when the output is redirected.

```
/* manual example (host): text from outside in another encoding, UTF-8 inside, back out.
 * expect: UTF-16LE with a 2-byte mark; 9 characters in 13 UTF-8 bytes; out as UTF-32BE in 36
bytes; strict refuses byte 2; code page 949 here: unsupported */
#include <gates/gates.h>
#include <proven/heap.h>

#include <stdio.h>

static const char *name(gates_encoding_kind_t k) {
    static const char *const names[] = { "UTF-8", "UTF-16LE", "UTF-16BE", "UTF-32LE",
"UTF-32BE", "a code page" };
```

```
    return names[k];
}

int main(void) {
    /* A file's bytes: a UTF-16LE mark, then "Hi, " and two Hangul syllables U+D55C U+AE00,
then "!". */
    static const unsigned char file[] = { 0xFF, 0xFE, 'H', 0, 'i', 0, ',', 0, ' ', 0, 0x5C,
0xD5, 0x00, 0xAE,
                                          '!', 0, '\r', 0, '\n', 0 };
    gates_encoding_t enc;
    gates_usize_t mark = gates_encoding_detect(file, sizeof file, &enc);
    printf("%s with a %u-byte mark; ", name(enc.kind), (unsigned)mark);

    /* In: one encoding inside the program from here on (the mark is skipped). */
    gates_u8 *text = nullptr;
    gates_usize_t size = 0;
    if (!gates_is_ok(gates_encoding_to_utf8(enc, file, sizeof file, 0, (gates_allocator_t){0},
&text, &size, nullptr))) {
        return 1;
    }
    gates_str_t s = { .ptr = text, .size = size };
    unsigned chars = 0;
    for (gates_u32 at = 0; at < size; chars++) at += gates_text_decode(s, at, &(gates_u32){0});
    printf("%u characters in %u UTF-8 bytes; ", chars, (unsigned)size);

    /* Out: whatever the other side wants. */
    gates_u8 *wide = nullptr;
    gates_usize_t wide_size = 0;
    if (!gates_is_ok(gates_encoding_from_utf8((gates_encoding_t){ .kind =
GATES_ENCODING_UTF32BE }, s, 0,
                                              (gates_allocator_t){0}, &wide, &wide_size,
nullptr))) {
        return 1;
    }
    printf("out as UTF-32BE in %u bytes; ", (unsigned)wide_size);

    /* Broken input: replaced with U+FFFD by default, or refused with where it broke. */
    gates_usize_t bad_at = 0;
    gates_u8 *junk = nullptr;
    gates_err_t err = gates_encoding_to_utf8((gates_encoding_t){ .kind = GATES_ENCODING_UTF8 },
"ok\xFF", 3,
                                             GATES_ENCODING_STRICT, (gates_allocator_t){0},
&junk, &size, &bad_at);
    printf("strict refuses byte %u; ", err == PROVEN_ERR_INVALID_ENCODING ? (unsigned)bad_at :
99u);

    /* Code pages (949 = Korean Windows, EUC-KR and more) need a converter: on Windows the app
     * installs the platform's; gates_codepage_converter_cp949() is a built-in one for 949 and
     * 51949. This program installs none, so the call says so. */
    err = gates_encoding_to_utf8((gates_encoding_t){ .kind = GATES_ENCODING_CODEPAGE, .codepage
= GATES_CODEPAGE_CP949 },
                                 "\xC7\xD1", 2, 0, (gates_allocator_t){0}, &junk, &size,
nullptr);
    printf("code page 949 here: %s\n", err == PROVEN_ERR_UNSUPPORTED ? "unsupported" :
"converted");

    gates_allocator_t heap = proven_heap_allocator();
    heap.free_fn(heap.ctx, text);
    heap.free_fn(heap.ctx, wide);
    return 0;
}
```

6.7 Many lines: the editor

`gates_editor_create` (gates/editor.h) makes a multi-line editor over a text buffer. It paints only the lines it shows, so a long file costs what the window costs. Arrows, Home/End, PageUp/PageDown and Ctrl with them move the caret (Shift selects; Shift+click extends, a double click takes a word, a triple click the line with its break), Enter keeps the text's own line ending, Tab moves focus unless the description asks for tabs to be typed, and Ctrl+A/C/X/V/Z/Y work as everywhere. A person's edits are undoable - a typing or deleting run is one step - and `gates_editor_modified` tells whether the text differs from what was set or saved. The program reads the text through `gates_editor_buffer` and changes it with `gates_editor_set_text` or `gates_editor_replace`; TEXT_CHANGED carries no text, so a long file is never copied into an event.

```
/* manual example (host): a multi-line editor - typing, lines, undo and the modified mark.
 * expect: 3 lines, caret on line 2; after undo: 2 lines, modified: no */
#include <gates/gates.h>

#include <stdio.h>

static void press(gates_tree_t *t, gates_key_t key, bool ctrl) {
    gates_key_event_t ev = { .key = key, .ctrl = ctrl, .down = true };
    (void)gates_input_key(t, &ev);
    ev.down = false;
    (void)gates_input_key(t, &ev);
}

static void type(gates_tree_t *t, const char *s) {
    for (; *s != 0; s++) (void)gates_input_char(t, (gates_u32)(unsigned char)*s);
}

int main(void) {
    gates_tree_t *t = nullptr;
    if (!gates_is_ok(gates_tree_create(&(gates_tree_desc_t){0}, &t))) return 1;
    gates_node_t notes;
    gates_editor_desc_t desc = { .rows = 8, .cols = 40 };
    if (!gates_is_ok(gates_editor_create(t, gates_tree_root(t), &desc, &notes)) ||
        !gates_is_ok(gates_editor_set_text(t, notes, GATES_STR("Shopping\nMilk"))) ||
        !gates_is_ok(gates_node_set_access_name(t, notes, GATES_STR("Notes"))) ||
        !gates_is_ok(gates_layout_run(t, (gates_size_t){ 360, 200 },
gates_text_backend_builtin()))) {
        return 1;
    }
    /* A person goes to the end, starts a new line and types. */
    gates_tree_set_focus(t, notes);
    press(t, GATES_KEY_END, true);
    press(t, GATES_KEY_ENTER, false);
    type(t, "Bread");
    const gates_text_buffer_t *text = gates_editor_buffer(t, notes);
    gates_u32 caret = 0;
    gates_editor_selection(t, notes, nullptr, &caret);
    printf("%u lines, caret on line %u; ", gates_text_buffer_line_count(text),
gates_text_buffer_line_of(text, caret));

    /* Ctrl+Z takes the typing back, then the new line: the text is as it was set. */
    press(t, GATES_KEY_Z, true);
    press(t, GATES_KEY_Z, true);
    printf("after undo: %u lines, modified: %s\n", gates_text_buffer_line_count(text),
           gates_editor_modified(t, notes) ? "yes" : "no");
    gates_tree_destroy(t);
    return 0;
}
```

The description turns on soft wrap (rows break after the last blank that fits; Up and Down then move by rows, Home and End go to the row's start and end before the line's, and a caret at the end of a row stays drawn there), a line number gutter and auto-indent. When Tab types tabs, Tab and Shift+Tab indent and unindent the selected lines and Ctrl+Tab moves focus on, so the keyboard is never trapped. Highlighting is the program's: `gates_editor_set_styles` maps style bytes to colours, and a styler (`gates_editor_set_styler`) is asked, just before painting, to style the lines about to show whose styles are stale - an edit makes its line stale again - so style work follows the view, not the length of the text. `gates_editor_find` selects the next match; marks (`gates_editor_mark_add`) keep a place while the text around it changes. An input method composes at the caret, drawn underlined in the text; the result arrives as one edit. Screen readers read the editor's text by character, word and line - a line as shown, a wrapped row (`gates_access_text_line`); a paragraph is a text line - with one rectangle per row for a range (`gates_access_text_rects`).

```
/* manual example (host): highlighting through a styler, and find.
 * expect: styled up to line 11 of 200; "TODO" found on line 150 */
#include <gates/gates.h>

#include <stdio.h>
#include <string.h>

enum { PLAIN, NUMBER, KEYWORD };

/* The program's highlighter: numbers and the word "TODO". It only styles the
 * range it is given - the editor asks for what is about to show. */
static gates_u32 styled_to;
static void highlight(void *user, gates_text_buffer_t *text, gates_u32 from, gates_u32 to) {
    (void)user;
    gates_text_buffer_set_style(text, from, to, PLAIN);
    for (gates_u32 i = from; i < to; i++) {
        gates_u8 c = gates_text_buffer_byte(text, i);
        if (c >= '0' && c <= '9') gates_text_buffer_set_style(text, i, i + 1, NUMBER);
    }
    gates_u32 at = from;
    while (gates_text_buffer_find(text, at, GATES_STR("TODO"), 0, &at) && at + 4 <= to) {
        gates_text_buffer_set_style(text, at, at + 4, KEYWORD);
        at += 4;
    }
    styled_to = to;
}

int main(void) {
    gates_tree_t *t = nullptr;
    if (!gates_is_ok(gates_tree_create(&(gates_tree_desc_t){0}, &t))) return 1;
    static char text[8000];
    int n = 0;
    for (int i = 1; i <= 200; i++) n += snprintf(text + n, sizeof text - (size_t)n, i == 150 ?
"TODO %d\n" : "step %d\n", i);
    gates_node_t ed;
    gates_editor_desc_t desc = { .rows = 10, .line_numbers = true, .wrap = true };
    const gates_editor_style_t styles[] = { {0}, { .token = GATES_COLOR_FOCUS_RING }, { .token
= GATES_COLOR_ERROR } };
    if (!gates_is_ok(gates_editor_create(t, gates_tree_root(t), &desc, &ed)) ||
        !gates_is_ok(gates_editor_set_text(t, ed, (gates_str_t){ (const gates_u8 *)text,
(gates_usize_t)n })) ||
        !gates_is_ok(gates_editor_set_styles(t, ed, styles, 3)) ||
        !gates_is_ok(gates_editor_set_styler(t, ed, highlight, nullptr)) ||
        !gates_is_ok(gates_layout_run(t, (gates_size_t){ 320, 200 },
gates_text_backend_builtin()))) {
        return 1;
```

```
    }
    /* Painting styles only the lines shown (and the next one). */
    gates_draw_list_t dl;
    if (!gates_is_ok(gates_draw_list_init(&dl, (gates_allocator_t){0}, 0)) ||
        !gates_is_ok(gates_paint_tree(t, &dl, gates_theme_light(),
gates_text_backend_builtin()))) {
        return 1;
    }
    const gates_text_buffer_t *buf = gates_editor_buffer(t, ed);
    printf("styled up to line %u of %u; ", gates_text_buffer_line_of(buf, styled_to),
gates_text_buffer_line_count(buf) - 1);

    /* Find selects the match and scrolls to it. */
    if (gates_editor_find(t, ed, GATES_STR("todo"), GATES_FIND_IGNORE_CASE, true)) {
        gates_u32 caret = 0;
        gates_editor_selection(t, ed, nullptr, &caret);
        printf("\"TODO\" found on line %u\n", gates_text_buffer_line_of(buf, caret) + 1);
    }
    gates_draw_list_deinit(&dl);
    gates_tree_destroy(t);
    return 0;
}
```

6.8 Large texts

`gates/text_buffer.h` holds UTF-8 text of any length a tool edits (up to 1 GiB): a gap buffer with an index of line starts, a style byte per byte, marks that move with edits, and search. Reading never moves the gap - a range comes back as at most two spans - so a program can read the lines it shows while editing elsewhere. The multi-line editor is built on it; a program can also use it alone to load, search and change a large text.

7 Workers and timers

In this chapter

7.1 Workers never touch the tree 34
7.2 Background tasks 36
7.3 Timers 38

Headers: `gates/post.h`, `gates/timer.h`.

7.1 Workers never touch the tree

The tree belongs to the UI thread. A worker thread - reading files, talking to a device, computing - hands its results over by posting messages. On the UI thread, before the worker starts, the program takes the app's sender (`gates_app_sender`) and builds a target: the tree and the node that should receive the results (`gates_target`). The worker posts messages carrying a kind and a payload; the UI thread delivers them at a safe point to the node's message handler (`gates_node_set_message_handler`) and then releases the payload.

```
/* manual example (windows): a worker thread feeds a log through the sender.
 * The worker never touches the tree; it posts lines, and the UI thread
 * delivers them to the log's message handler. */
#include <gates/gates.h>

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <windows.h>

enum { MSG_LINE = 1 };

typedef struct job_t {
    gates_sender_t *sender;      /* retained for the worker */
    gates_target_t target;       /* built on the UI thread */
} job_t;

static void release_line(void *payload, void *ctx) {
    (void)ctx;
    free(payload); /* may run on any thread: only frees */
}

static DWORD WINAPI worker(void *arg) {
    job_t *job = arg;
    for (int i = 1; i <= 1000; i++) {
        char *line = malloc(40);
        if (line == nullptr) break;
        int n = snprintf(line, 40, "line %d from the worker", i);
        gates_message_t msg = { .target = job->target, .kind = MSG_LINE, .payload = line,
                                .bytes = (gates_usize_t)n, .release = release_line };
        gates_err_t err = gates_sender_post(job->sender, &msg);
        if (err == GATES_POST_FULL) { free(line); Sleep(5); i--; continue; } /* the caller
keeps it */
        if (err == GATES_POST_CLOSED) { free(line); break; }                 /* the app is
closing */
```

```
        if (!gates_is_ok(err)) { free(line); break; }
    }
    gates_sender_release(job->sender);
    return 0;
}

static void on_line(gates_tree_t *tree, gates_node_t node, gates_u32 kind, void *payload,
                    gates_usize_t bytes, void *user) {
    (void)user;
    if (kind == MSG_LINE) (void)gates_log_append(tree, node, (gates_str_t){ .ptr =
payload, .size = bytes });
}

int main(void) {
    gates_app_t *app = nullptr;
    if (!gates_is_ok(gates_app_create(&(gates_app_desc_t){0}, &app))) return 1;
    gates_window_t *win = nullptr;
    gates_window_desc_t desc = { .title = GATES_STR("Worker"), .size = { 420, 300 } };
    if (!gates_is_ok(gates_window_create(app, &desc, nullptr, &win))) {
        gates_app_destroy(app);
        return 1;
    }
    gates_tree_t *t = gates_window_tree(win);
    gates_node_t root = gates_tree_root(t), log;
    job_t job = {0};
    gates_err_t err = gates_layout_set(t, root, GATES_LAYOUT_KIND_COLUMN);
    if (gates_is_ok(err)) err = gates_log_create(t, root, &(gates_log_desc_t){ .max_lines =
500 }, &log);
    if (gates_is_ok(err)) err = gates_layout_set_child_grow(t, log, 1);
    if (gates_is_ok(err)) err = gates_node_set_access_name(t, log, GATES_STR("Worker output"));
    if (gates_is_ok(err)) err = gates_node_set_message_handler(t, log, on_line, nullptr);
    if (gates_is_ok(err)) err = gates_app_sender(app, &job.sender);
    HANDLE thread = nullptr;
    if (gates_is_ok(err)) {
        job.target = gates_target(t, log);
        thread = CreateThread(nullptr, 0, worker, &job, 0, nullptr);
        if (thread == nullptr) gates_sender_release(job.sender);
    }
    if (gates_is_ok(err)) err = gates_app_run(app);
    gates_window_destroy(win);
    gates_app_destroy(app); /* closes the sender: the worker sees CLOSED and stops */
    if (thread != nullptr) {
        WaitForSingleObject(thread, INFINITE);
        CloseHandle(thread);
    }
    return gates_is_ok(err) ? 0 : 1;
}
```

The rules that make this safe:

- Posting never blocks and never allocates. A full queue answers `GATES_POST_FULL`: the worker keeps its payload and tries later, drops it, or merges it into the next one. A closed queue answers `GATES_POST_CLOSED`: the app is shutting down, stop and release the sender.
- When a post succeeds, gates owns the payload and releases it exactly once - after delivery, when the target node is gone, when a newer snapshot replaces it (`replaceable`), or at shutdown. When a post fails, the worker still owns it.
- Release functions may run on any thread; they only free.
- The queue is bounded (1024 messages and 1 MiB of payload by default, `gates_app_desc_t`), and at most 64 messages are delivered per turn, so a flood never starves input and painting. It starts

with 256 slots and grows on the UI thread as it fills, up to the limit, so a large limit costs memory only when bursts use it. Choose the limit by the delay you accept: a queue holds about limit / delivery-rate of waiting (at about 55000 messages a second, 1024 is under 20 ms; 65536 is over a second). No limit at all would only turn a flood into minutes of delay and unbounded memory: the delivery rate, not the queue, sets throughput.

- A sender stays valid on every thread until its last reference is released, even after the app is gone.

7.2 Background tasks

For the common case - one job with progress, a result and a Cancel button - `gates_task_start` (gates/task.h) does the posting for you. The work function runs on its own thread (the window supplies it), calls `gates_task_report` with a per-mille and a short text, looks at `gates_task_cancelled` now and then, and returns its result. On the UI thread `on_progress` shows the latest report (reports the UI has not shown yet are replaced, never queued up) and `on_done` comes once, with the result and whether it was cancelled; the thread is joined for you. Closing the window cancels running tasks and waits for their work to return, so a work function should look at `gates_task_cancelled` often.

```
/* manual example (windows): a background task counts primes while the window
 * shows its progress and a Cancel button stops it. The work never touches the
 * tree: it reports, polls for cancellation and returns; the UI hears both. */
#include <gates/gates.h>

#include <stdio.h>

#define LIMIT 3000000u

typedef struct count_t {
    gates_u32 primes;            /* written by the work, read in on_done */
    gates_node_t bar, status, cancel;
    gates_task_t *task;
} count_t;

static bool is_prime(gates_u32 n) {
    if (n < 2) return false;
    for (gates_u32 d = 2; d * d <= n; d++) {
        if (n % d == 0) return false;
    }
    return true;
}

/* On the worker thread. */
static gates_err_t count_primes(gates_task_t *task, void *user) {
    count_t *c = user;
    for (gates_u32 n = 0; n < LIMIT; n++) {
        if (is_prime(n)) c->primes++;
        if (n % (LIMIT / 100) == 0) {
            if (gates_task_cancelled(task)) return PROVEN_ERR_EOF;
            char text[48];
            int len = snprintf(text, sizeof text, "%u primes so far", c->primes);
            (void)gates_task_report(task, (gates_u32)((gates_u64)n * 1000 / LIMIT),
                                    (gates_str_t){ .ptr = (const gates_u8 *)text, .size =
(gates_usize_t)len });
        }
    }
    return GATES_OK;
}

/* On the UI thread. */
static void on_progress(gates_tree_t *tree, gates_task_t *task, gates_u32 permille, gates_str_t
```

```
text, void *user) {
    (void)task;
    count_t *c = user;
    (void)gates_progress_set_value(tree, c->bar, (gates_i32)permille);
    (void)gates_widget_set_text(tree, c->status, text);
}

static void on_done(gates_tree_t *tree, gates_task_t *task, gates_err_t result, bool cancelled,
void *user) {
    (void)task;
    count_t *c = user;
    char text[64];
    int len = cancelled ? snprintf(text, sizeof text, "Cancelled after %u primes", c->primes)
            : gates_is_ok(result) ? snprintf(text, sizeof text, "%u primes below %u", c-
>primes, LIMIT)
                                  : snprintf(text, sizeof text, "Stopped");
    (void)gates_widget_set_text(tree, c->status, (gates_str_t){ .ptr = (const gates_u8
*)text, .size = (gates_usize_t)len });
    (void)gates_widget_set_disabled(tree, c->cancel, true);
    c->task = nullptr;
}

static void on_cancel(gates_tree_t *tree, gates_node_t node, void *user) {
    (void)tree;
    (void)node;
    count_t *c = user;
    if (c->task != nullptr) gates_task_cancel(c->task);
}

int main(void) {
    gates_app_t *app = nullptr;
    if (!gates_is_ok(gates_app_create(&(gates_app_desc_t){0}, &app))) return 1;
    gates_window_t *win = nullptr;
    gates_window_desc_t desc = { .title = GATES_STR("Primes"), .size = { 380, 160 } };
    if (!gates_is_ok(gates_window_create(app, &desc, nullptr, &win))) {
        gates_app_destroy(app);
        return 1;
    }
    gates_tree_t *t = gates_window_tree(win);
    gates_node_t root = gates_tree_root(t);
    count_t c = {0};
    gates_err_t err = gates_layout_set(t, root, GATES_LAYOUT_KIND_COLUMN);
    if (gates_is_ok(err)) err = gates_layout_set_padding(t, root, 12);
    if (gates_is_ok(err)) err = gates_layout_set_gap(t, root, 8);
    if (gates_is_ok(err)) err = gates_progress_create(t, root, 0, &c.bar);
    if (gates_is_ok(err)) err = gates_node_set_access_name(t, c.bar, GATES_STR("Progress"));
    if (gates_is_ok(err)) err = gates_label_create(t, root, GATES_STR("Starting..."),
&c.status);
    if (gates_is_ok(err)) err = gates_button_create(t, root, GATES_STR("&Cancel"), on_cancel,
&c, &c.cancel);
    gates_task_desc_t task = { .work = count_primes, .work_user = &c, .on_progress =
on_progress,
                               .on_done = on_done, .ui_user = &c };
    if (gates_is_ok(err)) err = gates_task_start(t, &task, &c.task);
    if (gates_is_ok(err)) err = gates_app_run(app);
    gates_window_destroy(win); /* a task still running is cancelled and waited for */
    gates_app_destroy(app);
    return gates_is_ok(err) ? 0 : 1;
}
```

7.3 Timers

A timer belongs to a node and runs its callback on the UI thread after an interval, once or repeatedly. Cancelling is immediate: a cancelled timer never fires again, and destroying its node or tree cancels it. A late repeating timer fires once and is rescheduled from now, never in a burst. An idle window runs no platform timer at all.

A window's tree has a clock. A bare tree - in a test - gets one from the program, which then decides when time passes:

```
/* manual example (host): a repeating timer, driven by a test clock.
 * expect: 3 ticks, then cancelled */
#include <gates/gates.h>

#include <stdio.h>

static gates_u64 now_ms;
static gates_u64 clock_now(void *ctx) { (void)ctx; return now_ms; }

static void on_tick(gates_tree_t *tree, gates_node_t node, gates_timer_id_t id, void *user) {
    (void)node;
    int *ticks = user;
    if (++*ticks == 3) (void)gates_timer_cancel(tree, id); /* a callback may cancel itself */
}

int main(void) {
    gates_tree_t *t = nullptr;
    if (!gates_is_ok(gates_tree_create(&(gates_tree_desc_t){0}, &t))) return 1;
    /* A window's tree has a clock; a bare tree gets one from the program. */
    gates_tree_set_clock(t, clock_now, nullptr, nullptr);
    int ticks = 0;
    gates_timer_id_t id;
    if (!gates_is_ok(gates_timer_start(t, gates_tree_root(t), 100, true, on_tick, &ticks,
&id))) return 1;
    for (int step = 0; step < 10; step++) {
        now_ms += 100;
        (void)gates_tree_run_timers(t); /* the window does this when its timer fires */
    }
    printf("%d ticks, then %s\n", ticks, gates_timer_active(t, id) ? "still running" :
"cancelled");
    gates_tree_destroy(t);
    return 0;
}
```

Before animating, ask `gates_window_reduced_motion`: when the person has turned animation effects off, show the end state instead. app_settings does this with its "applying" step: a timer fills a progress bar, or the bar is simply full.

8 Themes, units and scaling

In this chapter

8.1 Logical units 39
8.2 Themes 39
8.3 Fonts 40
8.4 Drawing and text backends 40

Headers: `gates/theme.h`, `gates/geometry.h`, `gates/draw.h`, `gates/render.h`, `gates/text.h`.

8.1 Logical units

Every coordinate and size in the API is a logical unit, 1/96 inch: layout values, node rectangles, preferred sizes, window sizes, pointer positions. A window converts once, at its edge - to device pixels when it draws, back to units when it reports the pointer - so the same program is the same physical size on a 100 % and a 200 % display. `gates_px` and `gates_logical` are the conversions (exact inverses for whole units).

```
/* manual example (host): logical units and themes.
 * expect: 100 units = 150 px at 144 dpi; 150 px = 100 units; light theme: 0 issues */
#include <gates/gates.h>

#include <stdio.h>

int main(void) {
    /* Every coordinate in the API is a logical unit, 1/96 inch. A window
     * scales once, at its edge: to pixels when drawing, back when reading
     * pointer positions. */
    gates_u32 dpi = 144; /* 150 % */
    gates_i32 px = gates_px(100, dpi);
    gates_i32 back = gates_logical(px, dpi);
    /* A theme is checked against the enforced rules: contrast and cues. */
    gates_access_issue_t issues[8];
    gates_u32 n = gates_theme_audit(gates_theme_light(), issues, 8);
    printf("100 units = %d px at %u dpi; %d px = %d units; light theme: %u issues\n", px, dpi,
px, back, n);
    return n == 0 ? 0 : 1;
}
```

The window follows its monitor (per-monitor DPI, and moving between monitors), the Windows "Text size" setting, and an application zoom (`gates_window_set_zoom`, 25 % to 400 %). All of them scale the whole interface, as a magnified page does.

8.2 Themes

Colours are tokens (`gates_color_token_t`), not values: the program says "text", "focus", "error", and the theme says which colour that is. A window follows the system - light or dark mode, and high contrast from the system's own colours - and switches live when the person changes it; the title bar follows too. `gates_window_set_theme_mode` fixes a mode, `gates_window_set_theme` sets an application theme. Every theme is held to the contrast and cue rules of chapter 9: `gates_theme_audit` checks one.

Focus and errors are never shown by colour alone: the focus ring and the error border have a width (at least 2 units), so they read in any theme.

8.3 Fonts

Text uses one of two fonts (`gates_font_t` in `gates/text.h`):

Font	Is	On Windows
`GATES_FONT_UI` (the default)	the platform's UI face, proportional	the system message font: Segoe UI, Malgun Gothic on Korean Windows
`GATES_FONT_MONO`	fixed pitch, for code, logs and aligned columns	Consolas
a named face (0.12.0)	a face you name, from `gates_font_named`	that face at the UI size; the UI face when it is not installed

`gates_node_set_font` chooses the font of a node and, like CSS `font-family`, of everything under it that does not choose its own; `GATES_FONT_INHERIT` goes back to the parent's, and `gates_node_font` reads the effective one. Set it on a panel to switch a whole area, or on one view for a log. Dialogs and menus are not under the window's root: they start from the UI font unless you set one on them.

Faces by name (0.12.0) work two ways. For **one face everywhere**, give the application its UI face: `gates_app_desc_t.ui_font` at creation, or `gates_app_set_ui_font(app, name)` while it runs (a settings screen's font choice; it answers `PROVEN_ERR_NOT_FOUND` when the face is not installed, and an empty name goes back to the system's). Everything that uses `GATES_FONT_UI` - dialogs and menus too - takes it, at the system's UI size. For **a face per element**, register the name once with `gates_font_named(name, &face)` and set that face like `GATES_FONT_MONO`: on one node, on a panel for an area, or on the root for the whole window. The same name always gives the same face; up to `GATES_FONT_NAMED_MAX` (62) names. Names are the system's family names, in English or in the system's language ("Malgun Gothic" or its Korean name both work on Windows).

```
gates_app_desc_t desc = { .ui_font = GATES_STR("Malgun Gothic") };  /* one face for the program
*/
gates_font_t serif;
if (gates_is_ok(gates_font_named(GATES_STR("Georgia"), &serif))) {
    (void)gates_node_set_font(tree, quote, serif);                  /* one element */
}
```

Sizes (0.10.0) are a percentage of the system's text size, so they still follow the person's Windows "Text size" setting. `gates_node_set_font_size(tree, node, percent)` sets one from `GATES_FONT_SIZE_MIN` (50) to `GATES_FONT_SIZE_MAX` (400); `GATES_FONT_SIZE_SMALL` (85), `GATES_FONT_SIZE_LARGE` (125) and `GATES_FONT_SIZE_HEADING` (150) are the usual ones, and 0 goes back to the parent's. The size is inherited like the face but separately from it: a MONO view inside a heading panel is mono at 150 %. Everything the node measures and draws follows it - text, line and row heights, carets, a text box's `cols`. `gates_node_font_size` reads the effective size, and `gates_node_font` returns face and size together (`gates_font_face`, `gates_font_percent`).

```
(void)gates_node_set_font(tree, log_view, GATES_FONT_MONO);            /* aligned log lines */
(void)gates_node_set_font_size(tree, title, GATES_FONT_SIZE_HEADING); /* a page heading */
(void)gates_node_set_font_size(tree, note, GATES_FONT_SIZE_SMALL);    /* small print */
```

8.4 Drawing and text backends

A window paints its tree into a draw list (`gates/draw.h`) and the software renderer (`gates/render.h`) turns it into pixels. Text goes through a text backend (`gates/text.h`), which reports per font a line height, an average width (a sizing hint: a text box's `cols`) and the advance of every character; a

string's width is exactly the sum of its advances, and the backend draws each character at that offset. So layout, drawing, the caret and hit testing agree on every backend and at every scale. The builtin backend is fixed-pitch for both fonts, which keeps tests deterministic (a size scales its 8 x 16 cell; named faces draw as the UI one). A backend of your own reads the face with `gates_font_face(font)`, a named face's name with `gates_font_face_name(font)`, and scales by `gates_font_percent(font)`. An application can draw its own things in the window's paint callback; it never reads widget state there.

9 Accessibility

In this chapter

9.1 Every window is readable and usable without sight or pointer 42
9.2 Names 42
9.3 The enforced rules 43
9.4 Speaking up 43
9.5 Checking with Narrator 43

Header: `gates/access.h`.

9.1 Every window is readable and usable without sight or pointer

gates describes every node to assistive technology: a role (button, check box, edit, list, ...), a name, a description, states (focused, disabled, checked, selected, expanded, read-only, invalid, required, offscreen), a value, and the actions it accepts. Radio options, choice options, menu entries and view rows are items of their control, addressed by id. On Windows the window is a UI Automation provider, so Narrator and automation tools see all of it, and every action a person can take with the keyboard can be taken through UI Automation: invoke, toggle, select, expand, set a value, scroll, and read and select text.

Actions from assistive technology take the same paths as input: the same events, commands, limits and refusals. A screen reader cannot do what the keyboard could not.

9.2 Names

A control's accessible name comes, in this order, from: an explicit name (`gates_node_set_access_name`); a label tied to it (`gates_node_set_labelled_by`, like HTML's `<label for>`); its form label; its own text (a button's label, a check box's caption). A label placed next to a text box is not tied to it until the program says so - the audit finds such a field. A menu without a name of its own takes its menu bar title, the entry that opened it (a submenu), or the name of the control that had focus when the program opened it (a context menu) (0.10.0):

```
/* manual example (host): the audit finds a field without a name; a label fixes it.
 * expect: before: 1 issue (no name); after: 0 issues, name "City" */
#include <gates/gates.h>

#include <stdio.h>

static gates_u32 audit(gates_tree_t *t, gates_access_rule_t *first) {
    (void)gates_layout_run(t, (gates_size_t){ 300, 120 }, gates_text_backend_builtin());
    gates_access_issue_t issues[8];
    gates_u32 n = gates_access_audit(t, gates_theme_light(), issues, 8);
    if (n > 0) *first = issues[0].rule;
    return n;
}

int main(void) {
    gates_tree_t *t = nullptr;
    if (!gates_is_ok(gates_tree_create(&(gates_tree_desc_t){0}, &t))) return 1;
    gates_node_t root = gates_tree_root(t), caption, city;
```

```
    if (!gates_is_ok(gates_layout_set(t, root, GATES_LAYOUT_KIND_COLUMN)) ||
        !gates_is_ok(gates_label_create(t, root, GATES_STR("City"), &caption)) ||
        !gates_is_ok(gates_textbox_create(t, root, GATES_STR(""), 20, &city))) {
        return 1;
    }
    gates_access_rule_t rule = 0;
    gates_u32 before = audit(t, &rule);
    /* A label beside a field is not tied to it until the program says so. */
    if (!gates_is_ok(gates_node_set_labelled_by(t, city, caption))) return 1;
    gates_u32 after = audit(t, &rule);
    gates_access_info_t info;
    if (!gates_is_ok(gates_access_info(t, city, 0, &info))) return 1;
    printf("before: %u issue%s (%s); after: %u issues, name \"%.*s\"\n", before, before == 1 ?
"" : "s",
           rule == GATES_RULE_NO_NAME ? "no name" : "other", after, (int)info.name.size, (const
char *)info.name.ptr);
    gates_tree_destroy(t);
    return after == 0 ? 0 : 1;
}
```

A form field's help and error become its description, so a screen reader reads "Name, edit, required, enter a name". Automation ids (`gates_node_set_automation_id`) are stable names for test scripts; forms and commands give `field-<id>` and `cmd-<id>` by default.

9.3 The enforced rules

`gates_access_audit` checks a laid-out tree against rules that the examples and reference applications are held to:

Rule	Checks
NO_NAME	every interactive control has an accessible name
TARGET_SIZE	pointer targets are at least 24 x 24 units (WCAG 2.2, 2.5.8)
KEYBOARD	every interactive control can be reached with the keyboard
DUPLICATE_ID	automation ids are unique in a tree
CONTRAST	theme text and cues meet their contrast ratios
FOCUS_CUE	focus and error cues are at least 2 units wide

Run the audit in tests. On Windows, `GATES_ACCESS_STRICT=1` in the environment audits every window after each layout and shows the number of issues in its title.

9.4 Speaking up

A status line that changes as the person works should be a live region (`gates_node_set_live`): polite ones are read after the current speech, assertive ones at once. For a message that belongs to no control, `gates_window_announce` speaks it. A hidden system caret follows the text caret, so magnifiers keep the insertion point in view.

9.5 Checking with Narrator

Start Narrator (Ctrl+Windows+Enter), then use the program with the keyboard alone: every control should be announced with its role, name and state as focus reaches it, list rows as the selection moves, and errors when a submit fails. Narrator's focus rectangle should sit on the control that has focus.

10 Deployment and troubleshooting

In this chapter

10.1 Linking 44
10.2 Versions and compatibility 44
10.3 The application and its windows 44
10.4 Troubleshooting 45

Headers: `gates/app.h`, `gates/window.h`, `gates/version.h`.

10.1 Linking

Target	Link line
Windows program	`-lgates -lproven -lgdi32 -luser32 -limm32 -ldwmapi -ladvapi32 -luiautomationcore -lole32 -loleaut32 -luuid`, plus `-mwindows` for a program without a console
headless (any system)	`-lgates_core -lproven -lm`

Compile with a C23 compiler (`-std=c23`; GCC 14+, Clang 18+, or mingw-w64 with GCC 14+) and `-I<package>/include`. `examples/consumer/` in the package is a complete program with a Makefile that builds against the package alone. `make install PREFIX=/some/dir` in the source tree copies the headers and libraries to `PREFIX/include` and `PREFIX/lib`.

The result is one executable: the libraries are static, and gates needs nothing on the target machine beyond Windows 10 or 11 itself (UI Automation, the input method and DPI functions it uses are part of Windows; newer ones are looked up at run time).

10.2 Versions and compatibility

`gates/version.h` gives `GATES_VERSION_STRING` and `GATES_VERSION_NUMBER`; `gates_version()` says what the linked library is. Releases with the same minor version are source compatible with each other: a program that built against 0.2.0 builds against 0.2.1. 0.2.0 changed the text backend contract (a custom backend adds `glyph_advance` and takes a font instead of a size); programs that use gates' own backends only rebuild. 0.3.0 adds the application frame and changes one thing that existed: in button, check box and command labels, `&` before a letter or digit now marks a mnemonic and is not shown (write `&&` for a literal one). Releases are not promised to be binary compatible - public structs may change size - so rebuild the program with every update, and check at start-up:

```
if (gates_version() != GATES_VERSION_NUMBER) { /* headers and library differ */ }
```

10.3 The application and its windows

`gates_app_create` makes the application (normally one per program, on the UI thread, which it initialises as a COM single-threaded apartment for UI Automation); `gates_window_create` opens a window with its own tree; `gates_app_run` runs the loop until the last window closes or `gates_app_quit` is called; destroy windows, then the app. The allocator in `gates_app_desc_t` is used for everything gates allocates (the default is thread-safe).

10.4 Troubleshooting

Symptom	Cause
a handler never runs	no handler set, or the change was made by the program (setters are silent: use `gates_widget_notify`)
a call returns `PROVEN_ERR_INVALID_ARG` for a node you just used	the handle is stale: the node was destroyed (and maybe its slot reused)
`PROVEN_ERR_INVALID_STATE` from an action	the control is disabled, hidden, or behind a modal dialog
typed text does not appear	the text box is read-only, or the input passed its maximum length (LIMIT_EXCEEDED is waiting for an answer)
the IME does not open	the box is read-only or a password box
the interface is tiny or huge	sizes given in pixels instead of logical units (chapter 8)
worker results stop arriving	the worker ignored `GATES_POST_FULL` or kept posting after `GATES_POST_CLOSED`
Narrator reads "edit" with no name	the label beside the field is not tied to it (chapter 9)
duplicate symbols `proven_*` at link time	the program links two copies of proven: keep one

11 The application frame

In this chapter

11.1 Mnemonics 46
11.2 The menu bar 46
11.3 Keymap: listing and rebinding shortcuts 47
11.4 Toolbar 48
11.5 Status bar 49
11.6 Tooltips 49
11.7 Tabs 51
11.8 Keeping the arrangement 51
11.9 Accessibility 52

Headers: `gates/frame.h`, `gates/command.h`, `gates/state.h`.

A desktop tool usually has a frame around its content: a menu bar, keyboard access keys on its labels and buttons, and shortcuts a person can learn and change. All of it is driven by the commands of chapter 4, so one action keeps one label, one enabled state and one shortcut wherever it appears.

11.1 Mnemonics

In the text of a button, a check box, a command, a menu bar title or a tab, `&x` marks `x` as the mnemonic when `x` is a letter or digit: `"&Save"`, `"E&xit"`, `"Page &2"`. Alt+x then activates the control - a button is pressed, a check box toggled, a menu bar title opened. Two `&` show one (`"Fish && chips"`), and any other `&` is shown as it is, so `"Save & close"` needs no escaping. The mnemonic takes no room: `"&Save"` is exactly as wide as `"Save"`.

The letter is underlined while keyboard cues are visible, as in other Windows programs: from the moment Alt is pressed or the menu bar is entered from the keyboard until the next pointer press, or all the time when the "underline access keys" setting is on.

Labels often show data, such as file names, so a label parses `&` only when it has a target: `gates_label_set_target(tree, label, box)` makes Alt+x focus `box`. Give the box the label as its accessible name too (`gates_node_set_labelled_by`), and screen readers announce the key with the name.

When several controls share a letter, each Alt+x moves the focus to the next of them and activates nothing; the person presses Space or Enter. Only controls the keyboard can reach count: a disabled button, a hidden panel or the window behind a modal dialog is skipped. Mnemonics are ASCII letters and digits; a title in another script adds one in brackets after its text, `(&F)`, as Korean and Japanese Windows programs do.

11.2 The menu bar

A tree has at most one menu bar: `gates_menubar_create(tree, parent, scope, &bar)`. Each `gates_menubar_add(tree, bar, title, ids, count, nullptr)` adds a title over a list of commands of `scope`, where id 0 draws a separator - the same list a context menu takes. The menus are ordinary menu overlays, so everything chapter 4 says about menus holds: they close on Escape, an outside click or when the window loses focus, a chosen entry is checked again before its command runs, and a handler set on the open menu hears MENU_CLOSED.

With the pointer, a click on a title opens its menu below it; while one is open, moving over another title switches to it, and a click on the open title closes it. From the keyboard:

Keys	In menu mode (a title highlighted)	In an open menu
F10, or Alt pressed and released alone	enter / leave menu mode	leave
Left / Right	the previous / next title	the neighbouring menu, opened
Down, Up, Enter, Space	open the highlighted title	move / choose
a title's mnemonic letter	open that title	-
an entry's mnemonic letter	-	choose that entry
Escape	leave menu mode	back to the highlighted title

Alt+x with a title's letter opens that menu directly. Titles come before controls, so give a control a letter no title uses. F10 is the menu key only while a menu bar is reachable; in a window without one it stays an ordinary command shortcut. The bar is not a Tab stop, and a modal dialog makes it unreachable. Menus have no submenus in this version.

On Windows, gates takes Alt+letter only when it matches a mnemonic; Alt+F4, Alt+Tab and Alt+Space keep doing what Windows does.

11.3 Keymap: listing and rebinding shortcuts

The shortcut stored in each command is the keymap. `gates_command_count` and `gates_command_at` list a scope's commands (for a keymap editor or a help screen); `gates_command_set_shortcut` rebinds one and refuses a shortcut another command of the scope already has, naming that command in `*conflict`. `gates_shortcut_format` prints a shortcut as menus and screen readers show it (`"Ctrl+Shift+S"`, `"F5"`, `"Ctrl+Del"`), and `gates_shortcut_parse` reads the same text back, so a program can keep a person's keymap in a file of lines such as `save=Ctrl+S`. Rules from chapter 4 still hold: a shortcut needs Ctrl, or is a function key; Alt is never part of one.

```
/* manual example (host): a menu bar, its keyboard and its mnemonics.
 * expect: new 1, open 1; F10 highlights File; access key Alt+F; shortcut Ctrl+Shift+N */
#include <gates/gates.h>

#include <stdio.h>

enum { CMD_NEW = 1, CMD_OPEN, CMD_QUIT };

static void on_command(gates_tree_t *tree, gates_command_id_t id, void *user) {
    (void)tree;
    ((int *)user)[id]++;
}

static bool press(gates_tree_t *t, gates_key_t key) {
    gates_key_event_t e = { .key = key, .down = true };
    return gates_input_key(t, &e);
}

int main(void) {
    gates_tree_t *t = nullptr;
    if (!gates_is_ok(gates_tree_create(&(gates_tree_desc_t){0}, &t))) return 1;
    gates_node_t root = gates_tree_root(t), bar;
    int runs[4] = {0};
    /* "&" marks the mnemonic: Alt+N, and N while the menu is open. */
    gates_command_desc_t cmds[] = {
```

```
        { .id = CMD_NEW, .label = GATES_STR("&New"), .shortcut = { .letter = 'N', .ctrl =
true },
          .enabled = true, .invoke = on_command, .user = runs },
        { .id = CMD_OPEN, .label = GATES_STR("&Open..."), .enabled = true, .invoke =
on_command, .user = runs },
        { .id = CMD_QUIT, .label = GATES_STR("E&xit"), .enabled = true, .invoke =
on_command, .user = runs },
    };
    for (int i = 0; i < 3; i++) {
        if (!gates_is_ok(gates_command_register(t, root, &cmds[i]))) return 1;
    }
    static const gates_command_id_t file[] = { CMD_NEW, CMD_OPEN, 0, CMD_QUIT }; /* 0:
separator */
    if (!gates_is_ok(gates_layout_set(t, root, GATES_LAYOUT_KIND_COLUMN)) ||
        !gates_is_ok(gates_menubar_create(t, root, root, &bar)) ||
        !gates_is_ok(gates_menubar_add(t, bar, GATES_STR("&File"), file, 4, nullptr)) ||
        !gates_is_ok(gates_layout_run(t, (gates_size_t){ 400, 300 },
gates_text_backend_builtin()))) {
        return 1;
    }
    /* F10 enters menu mode, Down opens File on its first entry, Enter chooses it. */
    (void)press(t, GATES_KEY_F10);
    bool highlighted = gates_menubar_highlighted(t, bar) == 0;
    (void)press(t, GATES_KEY_DOWN);
    (void)press(t, GATES_KEY_ENTER);
    /* Alt+F opens File directly (the platform sends Alt+letter here); O chooses Open. */
    (void)gates_input_mnemonic(t, 'f');
    gates_key_event_t o = { .letter = 'O', .down = true };
    (void)gates_input_key(t, &o);
    (void)gates_tree_dispatch_events(t, 0);

    /* The keymap: rebind New, and print the shortcut as menus do. */
    gates_command_id_t conflict = 0;
    if (!gates_is_ok(gates_command_set_shortcut(t, root, CMD_NEW,
                                                (gates_shortcut_t){ .letter = 'N', .ctrl =
true, .shift = true },
                                                &conflict))) {
        return 1;
    }
    char keys[32];
    gates_shortcut_t now = gates_command_shortcut(t, root, CMD_NEW);
    (void)gates_shortcut_format(&now, keys, sizeof keys);

    gates_access_info_t info;
    if (!gates_is_ok(gates_access_info(t, bar, 1, &info))) return 1;   /* item 1: the File
title */
    printf("new %d, open %d; F10 %s File; access key %.*s; shortcut %s\n", runs[CMD_NEW],
runs[CMD_OPEN],
           highlighted ? "highlights" : "misses", (int)info.access_key.size, (const char
*)info.access_key.ptr,
           keys);
    gates_tree_destroy(t);
    return 0;
}
```

11.4 Toolbar

`gates_toolbar_create(tree, parent, scope, &bar)` makes a row of compact buttons, and each `gates_toolbar_add(tree, bar, id)` adds one bound to a command of `scope` (id 0 adds a separator). A button shows its command's label without the mnemonic markup, looks disabled while the

command is disabled, and looks pressed while the command is checked - a toggle such as Bold is a checked command (with the accent border, 0.14.0). Buttons are flat until the pointer is over them, on a band with a line under it, so a row of text buttons does not read as a row of tabs; an icon per button (`gates_command_set_icon`) sets them apart further.

A click invokes the command and leaves the keyboard focus where it was, so Cut and Paste buttons act on the text box the person is typing in. From the keyboard the toolbar is one Tab stop: Left, Right, Home and End move between its enabled buttons, and Space or Enter invokes the one with the focus. When the toolbar is narrower than its buttons, those that do not fit are left out and a >> button at its end lists their commands in a menu.

11.5 Status bar

`gates_statusbar_create(tree, parent, &bar)` makes the strip along the bottom of a window, and `gates_statusbar_add(tree, bar, text, grow, &segment)` adds a segment: a label, so its text changes with `gates_widget_set_text`. Segments are separated by thin lines; a segment with `grow` takes a share of the spare width, like a layout child. Status text is not announced on every change - make a segment a live region (`gates_node_set_live`) only when its news matters.

11.6 Tooltips

`gates_node_set_tooltip(tree, node, text)` gives a node a short help text. It shows in a small box below the node (above near the bottom of the window) after the pointer rests on it for `GATES_TOOLTIP_DELAY_MS` (500 ms), or that long after the keyboard focus arrives, and it hides on a press, a key, when the pointer or focus leaves, and after `GATES_TOOLTIP_SHOW_MS`. Moving from one tooltip to the next while one shows switches at once. Toolbar buttons have one without asking: the command's label and shortcut, `"Paste (Ctrl+V)"`. Tooltips never take input, and screen readers read the text as the node's help text; the shown tooltip is also a ToolTip element of its own, announced when it opens. A modal dialog opening hides it. They are timed by the tree's clock, and nothing is timed while no hovered or focused node has one.

```
/* manual example (host): a toolbar, a status bar and a tooltip.
 * expect: pasted 1, focus kept; tooltip "Paste (Ctrl+V)" after 500 ms; status Pasted */
#include <gates/gates.h>

#include <stdio.h>
#include <string.h>

enum { CMD_CUT = 1, CMD_PASTE };

typedef struct app_t {
    gates_node_t status;
    int pasted;
} app_t;

static void on_command(gates_tree_t *tree, gates_command_id_t id, void *user) {
    app_t *a = user;
    if (id == CMD_PASTE) {
        a->pasted++;
        (void)gates_widget_set_text(tree, a->status, GATES_STR("Pasted"));
    }
}

/* A test clock: a window's tree has a real one. */
static gates_u64 now_ms;
static gates_u64 clock_now(void *ctx) { (void)ctx; return now_ms; }
static void clock_changed(void *ctx) { (void)ctx; }
```

```
int main(void) {
    gates_tree_t *t = nullptr;
    if (!gates_is_ok(gates_tree_create(&(gates_tree_desc_t){0}, &t))) return 1;
    gates_tree_set_clock(t, clock_now, clock_changed, nullptr);
    gates_node_t root = gates_tree_root(t), bar, body, status_bar;
    app_t a = {0};
    gates_command_desc_t cmds[] = {
        { .id = CMD_CUT, .label = GATES_STR("Cu&t"), .shortcut = { .key = GATES_KEY_X, .ctrl =
true },
          .enabled = true, .invoke = on_command, .user = &a },
        { .id = CMD_PASTE, .label = GATES_STR("&Paste"), .shortcut = { .key =
GATES_KEY_V, .ctrl = true },
          .enabled = true, .invoke = on_command, .user = &a },
    };
    if (!gates_is_ok(gates_layout_set(t, root, GATES_LAYOUT_KIND_COLUMN)) ||
        !gates_is_ok(gates_command_register(t, root, &cmds[0])) ||
        !gates_is_ok(gates_command_register(t, root, &cmds[1])) ||
        !gates_is_ok(gates_toolbar_create(t, root, root, &bar)) ||
        !gates_is_ok(gates_toolbar_add(t, bar, CMD_CUT)) ||
        !gates_is_ok(gates_toolbar_add(t, bar, CMD_PASTE)) ||
        !gates_is_ok(gates_textbox_create(t, root, GATES_STR(""), 30, &body)) ||
        !gates_is_ok(gates_node_set_access_name(t, body, GATES_STR("Text"))) ||
        !gates_is_ok(gates_layout_set_child_grow(t, body, 1)) ||
        !gates_is_ok(gates_statusbar_create(t, root, &status_bar)) ||
        !gates_is_ok(gates_statusbar_add(t, status_bar, GATES_STR("Ready"), 1, &a.status)) ||
        !gates_is_ok(gates_layout_run(t, (gates_size_t){ 400, 300 },
gates_text_backend_builtin()))) {
        return 1;
    }
    gates_tree_set_focus(t, body);

    /* Find the Paste button through the accessibility model: item 2 of the bar. */
    gates_access_info_t info;
    if (!gates_is_ok(gates_access_info(t, bar, 2, &info))) return 1;
    gates_point_t p = { info.bounds.x + info.bounds.w / 2, info.bounds.y + info.bounds.h / 2 };

    /* Rest the pointer on it: the tooltip shows after GATES_TOOLTIP_DELAY_MS. */
    gates_pointer_event_t move = { .action = GATES_POINTER_MOVE, .pos = p };
    (void)gates_input_pointer(t, &move);
    now_ms += GATES_TOOLTIP_DELAY_MS;
    (void)gates_tree_run_timers(t);
    gates_str_t tip = {0};
    char tip_text[64] = "";
    if (gates_tooltip_shown(t, nullptr, nullptr, &tip, nullptr) && tip.size < sizeof tip_text)
{
        memcpy(tip_text, tip.ptr, tip.size);
    }

    /* A click invokes the command; the focus stays in the text box. */
    gates_pointer_event_t down = { .action = GATES_POINTER_DOWN, .button =
GATES_BUTTON_LEFT, .pos = p };
    gates_pointer_event_t up = { .action = GATES_POINTER_UP, .button = GATES_BUTTON_LEFT, .pos
= p };
    (void)gates_input_pointer(t, &down);
    (void)gates_input_pointer(t, &up);
    (void)gates_tree_dispatch_events(t, 0);

    gates_str_t s = gates_widget_text(t, a.status);
    printf("pasted %d, focus %s; tooltip \"%s\" after %u ms; status %.*s\n", a.pasted,
           gates_node_eq(gates_tree_focus(t), body) ? "kept" : "moved", tip_text,
```

```
GATES_TOOLTIP_DELAY_MS,
           (int)s.size, (const char *)s.ptr);
    gates_tree_destroy(t);
    return 0;
}
```

11.7 Tabs

`gates_tabs_create(tree, parent, &tabs)` makes a strip of titles over pages, and each `gates_tabs_add(tree, tabs, title, &page)` adds a tab and returns its page, a column panel for the controls. One page shows at a time; controls on the others are out of reach of the pointer, the keyboard and assistive technology. The selected tab and its page are one sheet (0.14.0): the page is framed, the selected tab opens into it with the accent along its top, and the other tabs are outlines behind it.

The strip is one Tab stop. Left, Right, Home and End select at once; Ctrl+Tab and Ctrl+Shift+Tab (or Ctrl+PgDn and Ctrl+PgUp) switch from anywhere inside the tabs and wrap around; a click selects and focuses the strip; a title's mnemonic selects its tab. When the focus was on the page that goes away, it moves to the first control of the new page, or to the strip when there is none. A person's switch reports GATES_EVENT_VALUE_CHANGED on the tabs node with `result` the new index; `gates_tabs_set_selected` changes it silently. When the titles do not fit, the strip shows whole titles - always the selected one - and a >> button; a press on it, or Alt+Down on the strip, opens a menu of every tab with the selected one checked.

11.8 Keeping the arrangement

A person who drags a split, picks a tab, widens a column or scrolls expects to find it so next time. `gates_state_save` writes those arrangements as text - `split`, `tabs`, `columns` and `scroll` lines - for every node that has an automation id, and `gates_state_load` applies such text to a newly built tree. The program chooses where the text lives, usually a file beside its settings. The id is the key, so keep ids the same between versions; lines that no longer match are skipped, so an old file never breaks a newer program. On Windows, `gates_window_placement` and `gates_window_set_placement` do the same for the window's position, size and maximized state, and bring a window whose monitor is gone back onto the nearest one.

```
/* manual example (host): tabs, and state kept between runs.
 * expect: tab 1 (Advanced) after Ctrl+Tab; saved 2 lines; next run: tab 1, split 700 */
#include <gates/gates.h>

#include <stdio.h>

/* Builds the same window each run: tabs and a split, both with automation ids. */
static bool build(gates_tree_t **out, gates_node_t *tabs, gates_node_t *split, gates_node_t
*apply) {
    gates_tree_t *t = nullptr;
    if (!gates_is_ok(gates_tree_create(&(gates_tree_desc_t){0}, &t))) return false;
    gates_node_t root = gates_tree_root(t), page, left, right, reset;
    bool ok = gates_is_ok(gates_layout_set(t, root, GATES_LAYOUT_KIND_COLUMN)) &&
              gates_is_ok(gates_tabs_create(t, root, tabs)) &&
              gates_is_ok(gates_tabs_add(t, *tabs, GATES_STR("&General"), &page)) &&
              gates_is_ok(gates_button_create(t, page, GATES_STR("Apply"), nullptr, nullptr,
apply)) &&
              gates_is_ok(gates_tabs_add(t, *tabs, GATES_STR("&Advanced"), &page)) &&
              gates_is_ok(gates_button_create(t, page, GATES_STR("Reset"), nullptr, nullptr,
&reset)) &&
              gates_is_ok(gates_node_set_automation_id(t, *tabs, GATES_STR("settings.tabs")))
```

```
&&
              gates_is_ok(gates_panel_create(t, root, split)) &&
              gates_is_ok(gates_layout_set(t, *split, GATES_LAYOUT_KIND_SPLIT)) &&
              gates_is_ok(gates_panel_create(t, *split, &left)) &&
              gates_is_ok(gates_panel_create(t, *split, &right)) &&
              gates_is_ok(gates_node_set_automation_id(t, *split, GATES_STR("main.split"))) &&
              gates_is_ok(gates_layout_run(t, (gates_size_t){ 400, 300 },
gates_text_backend_builtin()));
    *out = t;
    return ok;
}

int main(void) {
    gates_tree_t *t;
    gates_node_t tabs, split, apply;
    if (!build(&t, &tabs, &split, &apply)) return 1;
    /* The person works in General, then Ctrl+Tab switches to Advanced. */
    gates_tree_set_focus(t, apply);
    gates_key_event_t ctrl_tab = { .key = GATES_KEY_TAB, .ctrl = true, .down = true };
    (void)gates_input_key(t, &ctrl_tab);
    gates_u32 now = gates_tabs_selected(t, tabs);
    gates_str_t title = gates_tabs_title(t, tabs, now);
    /* ...and drags the split. Save the state (a real program writes it to a file). */
    if (!gates_is_ok(gates_layout_set_split(t, split, GATES_SPLIT_HORIZONTAL, 700))) return 1;
    gates_u8 text[256];
    gates_usize_t len = 0;
    if (!gates_is_ok(gates_state_save(t, text, sizeof text, &len))) return 1;
    int lines = 0;
    for (gates_usize_t i = 0; i < len; i++) lines += text[i] == '\n' && i > 16;
    gates_tree_destroy(t);

    /* The next run builds the window again and loads what was saved. */
    if (!build(&t, &tabs, &split, &apply)) return 1;
    if (!gates_is_ok(gates_state_load(t, (gates_str_t){ .ptr = text, .size = len }, nullptr)))
return 1;
    printf("tab %u (%.*s) after Ctrl+Tab; saved %d lines; next run: tab %u, split %d\n", now,
           (int)title.size - 1, (const char *)title.ptr + 1, lines, gates_tabs_selected(t,
tabs),
           gates_layout_split_ratio(t, split));
    gates_tree_destroy(t);
    return 0;
}
```

11.9 Accessibility

The menu bar is a MenuBar element whose titles are menu items with ExpandCollapse; in menu mode the highlighted title has the keyboard focus. Every control with a mnemonic reports it as its access key ("Alt+F"; an entry of an open menu reports its letter), menu entries and buttons bound to a command report the command's shortcut as their accelerator key, and accessible names never contain the `&` markup. The toolbar is a ToolBar element whose buttons are items (a separator is none; `>>` is an item called More while it is shown), and the status bar is a StatusBar element with its segments as text. The tab strip is a Tab element whose titles are TabItem items with SelectionItem, and each page is a group named after its tab.

12 Numbers, groups and layouts

In this chapter

12.1 One handler for many controls 53
12.2 Once per turn: deferred calls 53
12.3 Spin box and slider 53
12.4 Group box 55
12.5 Grid and wrap layouts 55
12.6 Accessibility 56

Headers: `gates/inputs.h`, `gates/event.h`, `gates/widget.h`, `gates/layout.h`.

12.1 One handler for many controls

A calculator has twenty buttons; a settings page has a dozen fields that all mean "something changed". `gates_node_set_bubble_handler(tree, node, fn, user)` puts one handler on a container

- a panel, a form, the root - and it hears the events of every descendant that has no handler

of its own, with `ev->source` naming the control. Only the nearest such container hears an event, and an overlay is a root of its own: a dialog's buttons never reach the window's handler, only a handler inside the dialog.

12.2 Once per turn: deferred calls

Some work should follow a burst of changes, not each of them: a total over many fields, a filter over a list. `gates_tree_defer(tree, key, fn, user)` asks for `fn` to run once at the next safe point - after the events queued now are delivered - however often it is asked before then; the key names the job (`gates_tree_cancel_defer` drops it). A job asked for while deferred jobs run waits for the next turn, so a job cannot keep the program busy.

12.3 Spin box and slider

Both edit an integer in a range (`gates_range_t`: `min`, `max`, `step`, `page`, `value`). A decimal quantity uses a `scale`: with scale 100 the value 125 is shown as `1.25`, so money and measurements never meet floating-point rounding. A person's change reports GATES_EVENT_VALUE_CHANGED with `ev->value`; `gates_range_set_value` changes the value silently and `gates_range_set` changes the limits.

`gates_spin_create` makes a text box with up/down arrows. Up and Down step, PgUp and PgDn page, and the arrows step with a click and repeat while held; Left, Right, Home and End stay with the text. The wheel steps a spin box or slider only while it has focus - unfocused, it scrolls the page, so reading a form never changes a value by accident. While typed text is not a number in range the box is marked invalid; Enter or leaving the box commits it, and text that is still not a number in range goes back to the value. `gates_spin_box` returns the text box, for a label's target or a width.

`gates_slider_create` makes a track with a thumb, horizontal or vertical (up is more). Arrows step, PgUp and PgDn page, Home and End go to the ends, dragging the thumb moves in whole steps, and a press on the track pages toward the pointer. `gates_slider_set_ticks` draws a mark every n steps.

`gates_range_format` and `gates_range_parse` convert values and text the way the spin box does (`,` is read as `.`).

```c
/* manual example (host): a spin box and a slider, one bubble handler, one deferred total.
 * expect: width 2.50, volume 30; the panel heard 2 reports, the total was computed 1 time:
2.50 x 30 = 75.00 */
#include <gates/gates.h>

#include <stdio.h>

typedef struct app_t {
    gates_node_t width, volume;
    int heard, totals;
    char total[128];
} app_t;

/* Runs once after all the changes of a turn, however many there were. */
static void recompute(gates_tree_t *tree, gates_u32 key, void *user) {
    (void)key;
    app_t *a = user;
    a->totals++;
    char w[32];
    gates_i64 wv = gates_range_value(tree, a->width), vv = gates_range_value(tree, a->volume);
    (void)gates_range_format(wv, 100, w, sizeof w);
    char t[32];
    (void)gates_range_format(wv * vv, 100, t, sizeof t);
    snprintf(a->total, sizeof a->total, "%s x %lld = %s", w, (long long)vv, t);
}

/* One handler on the panel hears every control in it (ev->source says which). */
static void on_change(gates_tree_t *tree, const gates_event_t *ev, void *user) {
    app_t *a = user;
    if (ev->kind != GATES_EVENT_VALUE_CHANGED) return;
    a->heard++;
    (void)gates_tree_defer(tree, 1, recompute, a);
}

static bool key(gates_tree_t *t, gates_key_t k) {
    gates_key_event_t e = { .key = k, .down = true };
    return gates_input_key(t, &e);
}

int main(void) {
    gates_tree_t *t = nullptr;
    if (!gates_is_ok(gates_tree_create(&(gates_tree_desc_t){0}, &t))) return 1;
    gates_node_t root = gates_tree_root(t);
    app_t a = {0};
    /* Width in hundredths: 1.50 to 5.00 in steps of 0.25. */
    gates_range_t width = { .min = 150, .max = 500, .step = 25, .value = 200, .scale = 100 };
    gates_range_t volume = { .min = 0, .max = 100, .step = 5, .page = 20, .value = 10 };
    if (!gates_is_ok(gates_layout_set(t, root, GATES_LAYOUT_KIND_COLUMN)) ||
        !gates_is_ok(gates_spin_create(t, root, &width, &a.width)) ||
        !gates_is_ok(gates_slider_create(t, root, &volume, false, &a.volume)) ||
        !gates_is_ok(gates_node_set_bubble_handler(t, root, on_change, &a)) ||
        !gates_is_ok(gates_layout_run(t, (gates_size_t){ 400, 200 },
gates_text_backend_builtin()))) {
        return 1;
    }
    /* The person: two steps up in the spin box, a page up on the slider. */
    gates_tree_set_focus(t, gates_spin_box(t, a.width));
    (void)key(t, GATES_KEY_UP);
    (void)key(t, GATES_KEY_UP);
    gates_tree_set_focus(t, a.volume);
    (void)key(t, GATES_KEY_PAGE_UP);
```

```
    (void)gates_tree_dispatch_events(t, 0);

    char w[32];
    (void)gates_range_format(gates_range_value(t, a.width), 100, w, sizeof w);
    /* The spin box's two steps coalesce into one report of the latest value. */
    printf("width %s, volume %lld; the panel heard %d reports, the total was computed %d time:
%s\n", w,
           (long long)gates_range_value(t, a.volume), a.heard, a.totals, a.total);
    gates_tree_destroy(t);
    return 0;
}
```

12.4 Group box

`gates_group_create(tree, parent, title, collapsible, &group, &content)` draws a titled frame around a column panel; put related controls in `content`. The title takes mnemonic markup: in a plain group Alt+x goes to the first control inside, like a label with a target. A collapsible group's title is a Tab stop with an open/closed mark: Space, Enter, a click or its mnemonic shows or hides the content, whose controls then take no room and cannot be reached. A person's toggle reports VALUE_CHANGED on the group with `ev->checked` = expanded; `gates_group_set_expanded` is silent.

12.5 Grid and wrap layouts

`GATES_LAYOUT_KIND_GRID` puts children in rows of `gates_layout_set_grid` columns (two by default): each column as wide as its widest child, each row as tall as its tallest, children centred in their row and filling the cell's width unless their align asks for their own width. `gates_layout_set_grid_column_grow` gives spare width to columns - the usual choice is labels in a fixed column and fields in a growing one - `gates_layout_set_grid_row_grow` gives spare height to rows (the first `GATES_GRID_MAX_GROW_ROWS`), and `gates_layout_set_child_span` lets a child cover several columns. `GATES_LAYOUT_KIND_WRAP` places children left to right at their own size and starts a new line when the next does not fit, like words on a page; its height follows the width it is given. The gap applies both ways in both.

```
/* manual example (host): a grid form inside a collapsible group, and wrapped chips.
 * expect: labels in column 1, fields in column 2 at x 40; 7 chips on 3 lines; folded: 0 fields
reachable */
#include <gates/gates.h>

#include <stdio.h>

int main(void) {
    gates_tree_t *t = nullptr;
    if (!gates_is_ok(gates_tree_create(&(gates_tree_desc_t){0}, &t))) return 1;
    gates_node_t root = gates_tree_root(t), group, content, grid, chips, field[2], chip[7];
    const gates_text_backend_t *be = gates_text_backend_builtin(); /* 8 units a character */
    bool ok = gates_is_ok(gates_layout_set(t, root, GATES_LAYOUT_KIND_COLUMN)) &&
              gates_is_ok(gates_group_create(t, root, GATES_STR("&Connection"), true, &group,
&content)) &&
              gates_is_ok(gates_panel_create(t, content, &grid)) &&
              gates_is_ok(gates_layout_set(t, grid, GATES_LAYOUT_KIND_GRID)) &&   /* two
columns */
              gates_is_ok(gates_layout_set_gap(t, grid, 8)) &&
              gates_is_ok(gates_layout_set_grid_column_grow(t, grid, 1, 1));    /* fields take
the rest */
    static const char *names[2] = { "Host", "Port" };
    for (int i = 0; ok && i < 2; i++) {
        gates_node_t l;
```

```
        ok = gates_is_ok(gates_label_create(t, grid, (gates_str_t){ .ptr = (const gates_u8
*)names[i], .size = 4 }, &l)) &&
             gates_is_ok(gates_textbox_create(t, grid, GATES_STR(""), 12, &field[i])) &&
             gates_is_ok(gates_node_set_labelled_by(t, field[i], l));
    }
    ok = ok && gates_is_ok(gates_panel_create(t, root, &chips)) &&
         gates_is_ok(gates_layout_set(t, chips, GATES_LAYOUT_KIND_WRAP)) &&
         gates_is_ok(gates_layout_set_gap(t, chips, 4));
    for (int i = 0; ok && i < 7; i++) {
        ok = gates_is_ok(gates_button_create(t, chips, GATES_STR("tag"), nullptr, nullptr,
&chip[i]));
    }
    if (!ok || !gates_is_ok(gates_layout_run(t, (gates_size_t){ 160, 400 }, be))) return 1;

    gates_rect_t g = gates_node_layout_rect(t, grid), f = gates_node_layout_rect(t, field[0]);
    int lines = 1;
    for (int i = 1; i < 7; i++) {
        lines += gates_node_layout_rect(t, chip[i]).y > gates_node_layout_rect(t, chip[i -
1]).y;
    }
    /* Folding the group hides its content: nothing inside takes room or focus. */
    if (!gates_is_ok(gates_group_set_expanded(t, group, false)) ||
        !gates_is_ok(gates_layout_run(t, (gates_size_t){ 160, 400 }, be))) {
        return 1;
    }
    int shown = gates_widget_focusable(t, field[0]) + gates_widget_focusable(t, field[1]);
    printf("labels in column 1, fields in column 2 at x %d; 7 chips on %d lines; folded: %d
fields reachable\n",
           f.x - g.x, lines, shown);
    gates_tree_destroy(t);
    return 0;
}
```

12.6 Accessibility

A spin box is a Spinner with a RangeValue in the units it shows (1.25 for a scaled value of 125, with the step and page as its small and large change), and its text box is an edit named after it; a slider is a Slider with a RangeValue. Screen readers and automation can set either value; it is clamped into the range and reported like a person's change. A group box is a Group named by its title; a collapsible one has ExpandCollapse, and its title's focus is the group's.

13 Images and native dialogs

In this chapter

13.1 Images 57
13.2 Icons 57
13.3 Native dialogs 59

Headers: `gates/image.h`, `gates/draw.h`, `gates/window.h`.

13.1 Images

A tree keeps images by id. `gates_image_add_rgba(tree, w, h, pixels, stride, &id)` copies straight-alpha RGBA pixels in; `gates_image_load_file` and `gates_image_load_memory` decode PNG, JPEG, BMP, GIF, ICO or TIFF through the decoder the platform installs (every Windows window has one; a bare tree answers UNSUPPORTED). `gates_image_remove` frees an image; nodes that showed it then show nothing, and ids are never reused.

An image's natural size is its pixel size in logical units, so a 32 x 16 picture covers 32 x 16 units and scales with the window like text: at 150 % it is drawn on 48 x 24 pixels, smoothed (bilinear). Supply larger pixels when a picture must stay sharp at large scales.

An image can carry those larger pixels itself (0.10.0). `gates_image_add_variant_rgba(tree, id, w, h, rgba, stride)` adds the same picture at another pixel size - a 16-unit icon drawn at 24 and 32 pixels, say - and `gates_image_load_variant_file` / `_memory` decode one. The natural size stays the first set's. Whenever the image is drawn, the renderer takes the smallest set that covers the device pixels it fills, else the largest: at 100 % the 16-pixel set, at 150 % the 24, at 200 % the 32. A set must have the picture's shape (its height within a pixel of w x natural h / natural w); one as wide as a set added before replaces it. `gates_image_variant_count`, `gates_image_variant_size` and `gates_image_pick_size` tell what an image holds and which set a size would use.

`gates_image_create(tree, parent, id, &node)` shows an image. `gates_image_node_set_size` gives it another size; the picture then keeps its aspect ratio, centred. An image with an accessible name (`gates_node_set_access_name`) is an Image to screen readers; without one it is decoration and not announced.

13.2 Icons

Icons are drawn at 16 x 16 units (`GATES_ICON_SIZE`). `gates_button_set_icon` puts one left of a button's label; a button with an icon and no text is an icon button, and needs an access name. `gates_command_set_icon` gives a command an icon: menus draw it in the gutter (a checked command shows its mark instead) and toolbars left of the label, or alone with `gates_toolbar_set_icons_only` - the label then lives on in the tooltip and the accessible name.

For custom painting, `gates_tree_image` returns the image for `gates_draw_image(dl, rect, image)`; the draw list borrows it, so keep it until the list is rendered.

```
/* manual example (host): an image from pixels, an icon on a button and on a command.
 * expect: logo 32x16, drawn 32x16 at 192 dpi as 64x32 pixels; Save button 20 units wider with
its icon */
#include <gates/gates.h>
```

```
#include <stdio.h>

int main(void) {
    gates_tree_t *t = nullptr;
    if (!gates_is_ok(gates_tree_create(&(gates_tree_desc_t){0}, &t))) return 1;
    gates_node_t root = gates_tree_root(t), logo, save;
    /* 32 x 16 pixels, a left-to-right fade from blue to transparent. */
    static gates_u8 px[16][32][4];
    for (int y = 0; y < 16; y++) {
        for (int x = 0; x < 32; x++) {
            px[y][x][0] = 0; px[y][x][1] = 90; px[y][x][2] = 200; px[y][x][3] = (gates_u8)(255
- x * 8);
        }
    }
    /* A 16 x 16 icon: a grey square. */
    static gates_u8 ic[16][16][4];
    for (int y = 0; y < 16; y++) {
        for (int x = 0; x < 16; x++) {
            ic[y][x][0] = ic[y][x][1] = ic[y][x][2] = 90; ic[y][x][3] = 255;
        }
    }
    gates_image_id_t pic = 0, icon = 0;
    if (!gates_is_ok(gates_image_add_rgba(t, 32, 16, &px[0][0][0], 0, &pic)) ||
        !gates_is_ok(gates_image_add_rgba(t, 16, 16, &ic[0][0][0], 0, &icon)) ||
        !gates_is_ok(gates_layout_set(t, root, GATES_LAYOUT_KIND_COLUMN)) ||
        !gates_is_ok(gates_image_create(t, root, pic, &logo)) ||
        !gates_is_ok(gates_node_set_access_name(t, logo, GATES_STR("Logo"))) ||   /* named: an
Image */
        !gates_is_ok(gates_layout_set_child_align(t, logo, GATES_ALIGN_START_V)) ||
        !gates_is_ok(gates_button_create(t, root, GATES_STR("Save"), nullptr, nullptr, &save))
||
        !gates_is_ok(gates_layout_set_child_align(t, save, GATES_ALIGN_START_V)) ||
        !gates_is_ok(gates_layout_run(t, (gates_size_t){ 200, 100 },
gates_text_backend_builtin()))) {
        return 1;
    }
    gates_i32 plain = gates_node_preferred_size(t, save).w;
    if (!gates_is_ok(gates_button_set_icon(t, save, icon)) ||
        !gates_is_ok(gates_layout_run(t, (gates_size_t){ 200, 100 },
gates_text_backend_builtin()))) {
        return 1;
    }
    /* Render at 192 dpi: the logo covers 64 x 32 device pixels. */
    gates_draw_list_t dl;
    static gates_u32 pixels[200 * 2 * 100 * 2];
    gates_pixels_t target = { .ptr = pixels, .w = 400, .h = 200, .stride_bytes = 1600 };
    if (!gates_is_ok(gates_draw_list_init(&dl, (gates_allocator_t){0}, 0)) ||
        !gates_is_ok(gates_paint_tree(t, &dl, gates_theme_light(),
gates_text_backend_builtin())) ||
        !gates_is_ok(gates_render_soft_scaled(&dl, target, gates_text_backend_builtin(), 192)))
{
        return 1;
    }
    gates_rect_t r = gates_node_layout_rect(t, logo), dev = gates_rect_px(r, 192);
    printf("logo %dx%d, drawn %dx%d at 192 dpi as %dx%d pixels; Save button %d units wider with
its icon\n",
           gates_image_size(t, pic).w, gates_image_size(t, pic).h, r.w, r.h, dev.w, dev.h,
           gates_node_preferred_size(t, save).w - plain);
    gates_draw_list_deinit(&dl);
    gates_tree_destroy(t);
```

```
    return 0;
}
```

13.3 Native dialogs

Choosing a file, a folder or a colour, and asking a yes-or-no question, are best left to the platform: people know its dialogs, and they come with the platform's keyboard support, screen reader support and recent places. On Windows:

- `gates_window_open_file`, `gates_window_save_file` and `gates_window_choose_folder` take a `gates_file_dialog_t` (a title, filters such as `"Pictures|*.png;*.jpg|All files|*.*"`, a starting folder, and for saving a suggested name) and write the chosen path, in UTF-8, to your buffer. A save dialog asks before overwriting. `gates_window_open_files` lets the person pick several files: each path ends with a NUL byte in the buffer, and `count` says how many.
- `gates_window_choose_color` starts from a colour and answers with the chosen one.
- `gates_window_message` shows a message with OK, OK/Cancel, Yes/No or Yes/No/Cancel and an information, warning, error or question icon, and returns the answer (Escape and the close box answer Cancel).

These are the platform's modal dialogs: the call returns when the person answers, and the window's own menus close first. A cancel is not an error - the path comes back empty, or `chosen` false. Call them from a command or event handler, never while painting.